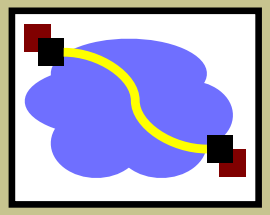


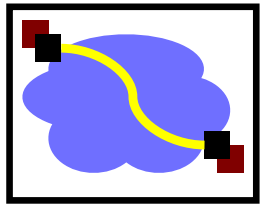
15-440 Distributed Systems

Review



Naming

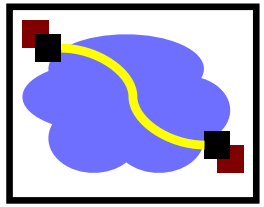
Names



- Names are associated with objects
  - Enables passing of references to objects
  - Indirection
  - Deferring decision on meaning/binding

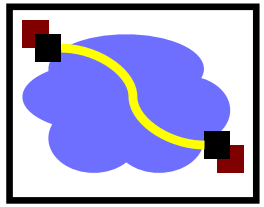
- Examples
  - Registers → R5
  - Memory → 0xdeadbeef
  - Host names → srini.com
  - User names → sseshan
  - Email → srini@cmu.edu
  - File name → /usr/srini/foo.txt
  - URLs → http://www.srini.com/index.html
  - Ethernet → f8:e4:fb:bf:3d:a6

Domain Name System Goals



- Basically a wide-area distributed database
- Scalability
- Decentralized maintenance
- Robustness
- Global scope
  - Names mean the same thing everywhere
- Don’t need
  - Atomicity
  - Strong consistency

DNS Records



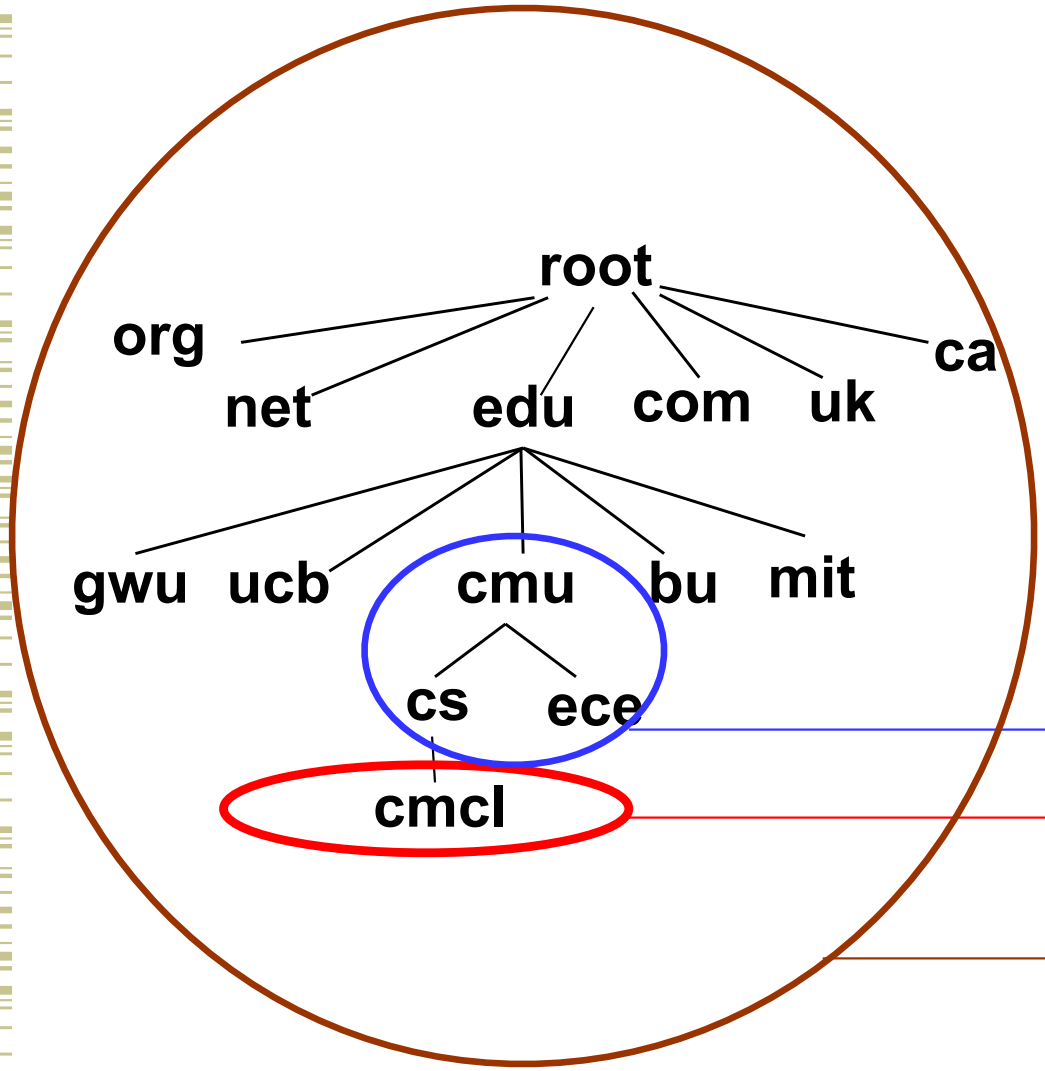
RR format: **(class, name, value, type, ttl)**

- DB contains tuples called resource records (RRs)
  - Classes = Internet (IN), Chaosnet (CH), etc.
  - Each class defines value associated with type

FOR IN class:

- Type=A
  - **name** is hostname
  - **value** is IP address
- Type=NS
  - **name** is domain (e.g. foo.com)
  - **value** is name of authoritative name server for this domain
- Type=CNAME
  - **name** is an alias name for some "canonical" (the real) name
  - **value** is canonical name
- Type=MX
  - **value** is hostname of mailserver associated with **name**

DNS Design: Zone Definitions



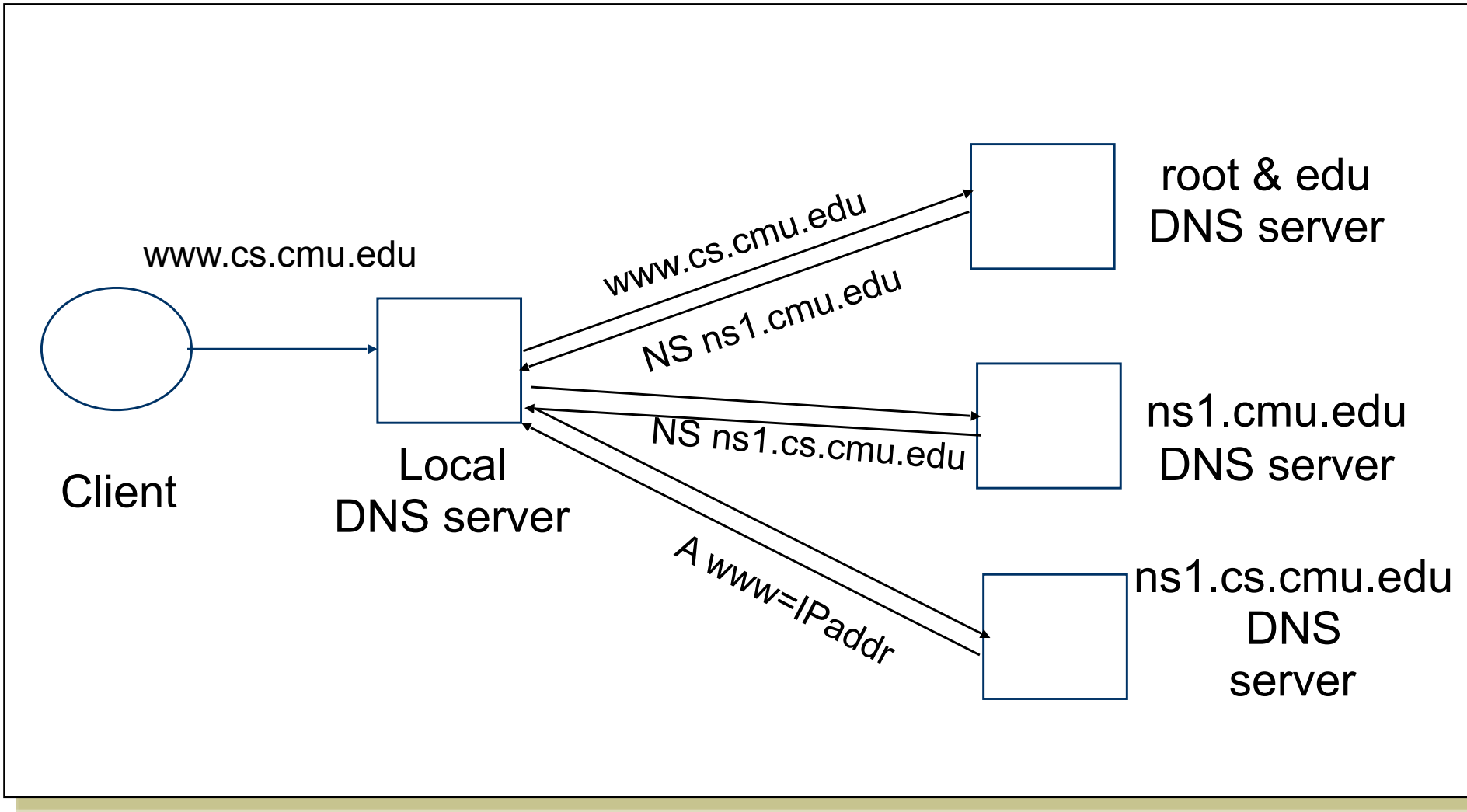
- Zone = contiguous section of name space
  - E.g., Complete tree, single node or subtree
- A zone has an associated set of name servers
  - Must store list of names and tree links

Physical Root Name Servers

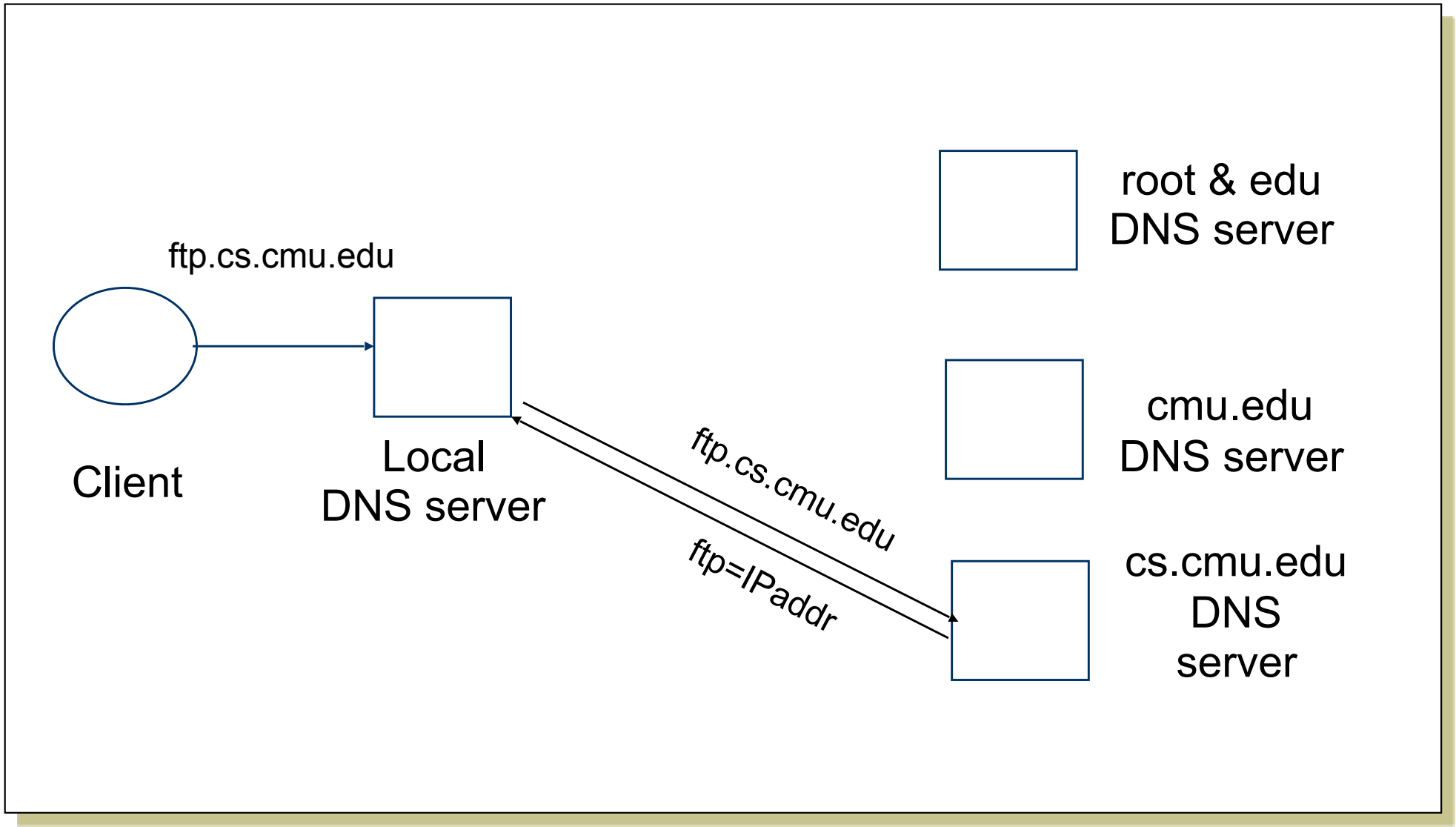


- Several root servers have multiple physical servers
- Packets routed to “nearest” server by “Anycast” protocol
- 346 servers total

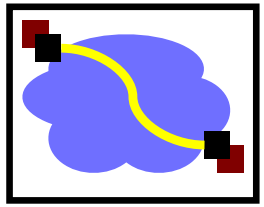
Typical Resolution



Subsequent Lookup Example

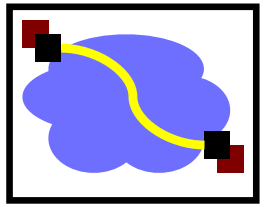


Prefetching



- Name servers can add additional data to response
- Typically used for prefetching
  - CNAME/MX/NS typically point to another host name
  - Responses include address of host referred to in “additional section”

Tracing Hierarchy (3 & 4)



- 3 servers handle CMU CS names

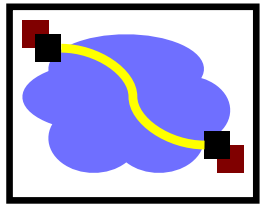
```
unix> dig +norecurse @nsauth1.net.cmu.edu NS greatwhite.ics.cs.cmu.edu

;; AUTHORITY SECTION:
cs.cmu.edu.      600     IN      NS      AC-DDNS-2.NET.cs.cmu.edu.
cs.cmu.edu.      600     IN      NS      AC-DDNS-1.NET.cs.cmu.edu.
cs.cmu.edu.      600     IN      NS      AC-DDNS-3.NET.cs.cmu.edu.
```

```
unix>dig +norecurse @AC-DDNS-2.NET.cs.cmu.edu NS
greatwhite.ics.cs.cmu.edu

;; AUTHORITY SECTION:
cs.cmu.edu.      300     IN      SOA   PLANISPHERE.FAC.cs.cmu.edu.
```

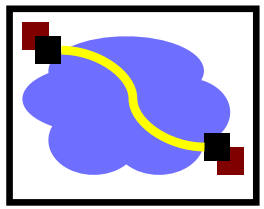
Hashing



Two uses of hashing that are becoming wildly popular in distributed systems:

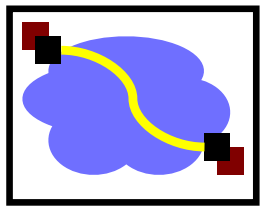
- Content-based naming
- Consistent Hashing of various forms

Consistent Hash

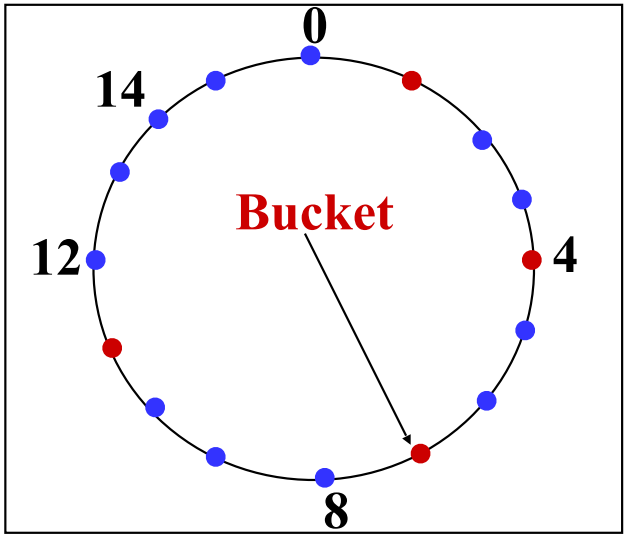


- “view” = subset of all hash buckets that are visible
- Desired features
  - Balanced – in any one view, load is equal across buckets
  - Smoothness – little impact on hash bucket contents when buckets are added/removed
  - Spread – small set of hash buckets that may hold an object regardless of views
  - Load – across all views # of objects assigned to hash bucket is small

Consistent Hash – Example

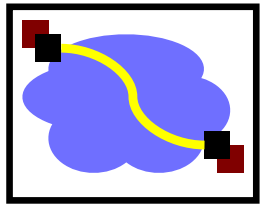


- Construction
  - Assign each of C hash buckets to random points on mod 2^n circle, where, hash key size = *n.*
  - Map object to random position on circle
  - Hash of object = closest clockwise bucket



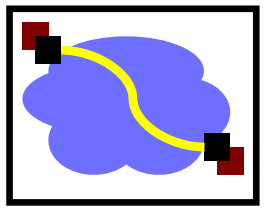
- Smoothness → addition of bucket does not cause much movement between existing buckets
- Spread & Load → small set of buckets that lie near object
- Balance → no bucket is responsible for large number of objects

Name items by their hash



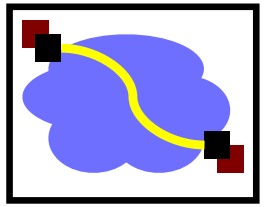
- Imagine that your filesystem had a layer of indirection:
  - pathname → hash(data)
  - hash(data) → list of blocks
- For example:
  - /src/foo.c -> 0xfff32f2fa11d00f0
  - 0xfff32f2fa11d00f0 -> [5623, 5624, 5625, 8993]
- If there were two identical copies of foo.c on disk ... We'd only have to store it once!
  - Name of second copy can be different

Self-Certifying Names



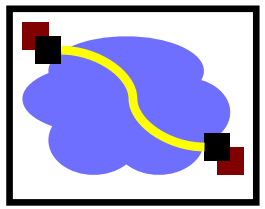
- Several p2p systems operate something like:
- Search for “national anthem”, find a particular file name (starspangled.mp3).
- Identify the files by the hash of their content (0x2fab4f001...)
- Request to download a file whose hash matches the one you want
- Advantage? You can verify what you got, even if you got it from an untrusted source (like some dude on a p2p network)

Hash functions

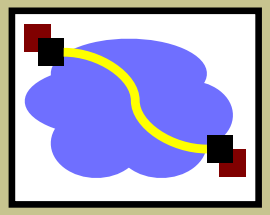


- Given a universe of possible objects U, map N objects from U to an M-bit hash.
- Typically, $|U| >>> 2^M$.
  - This means that there can be collisions: Multiple objects map to the same M-bit representation.
- Likelihood of collision depends on hash function, M, and N.
  - Birthday paradox → roughly 50% collision with $2^{M/2}$ objects for a well designed hash function

Desirable Properties (Cryptographic Hashes)

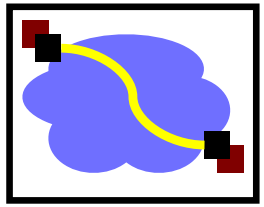


- Compression: Maps a variable-length input to a fixed-length output
- Ease of computation: A relative metric...
- Pre-image resistance:
  - Given a hash value h it should be difficult to find any message m such that h = hash(m)
- 2nd pre-image resistance:
  - Given an input m_1 it should be difficult to find different input m_2 such that hash(m_1) = hash(m_2)
- collision resistance:
  - difficult to find two different messages m_1 and m_2 such that hash(m_1) = hash(m_2)

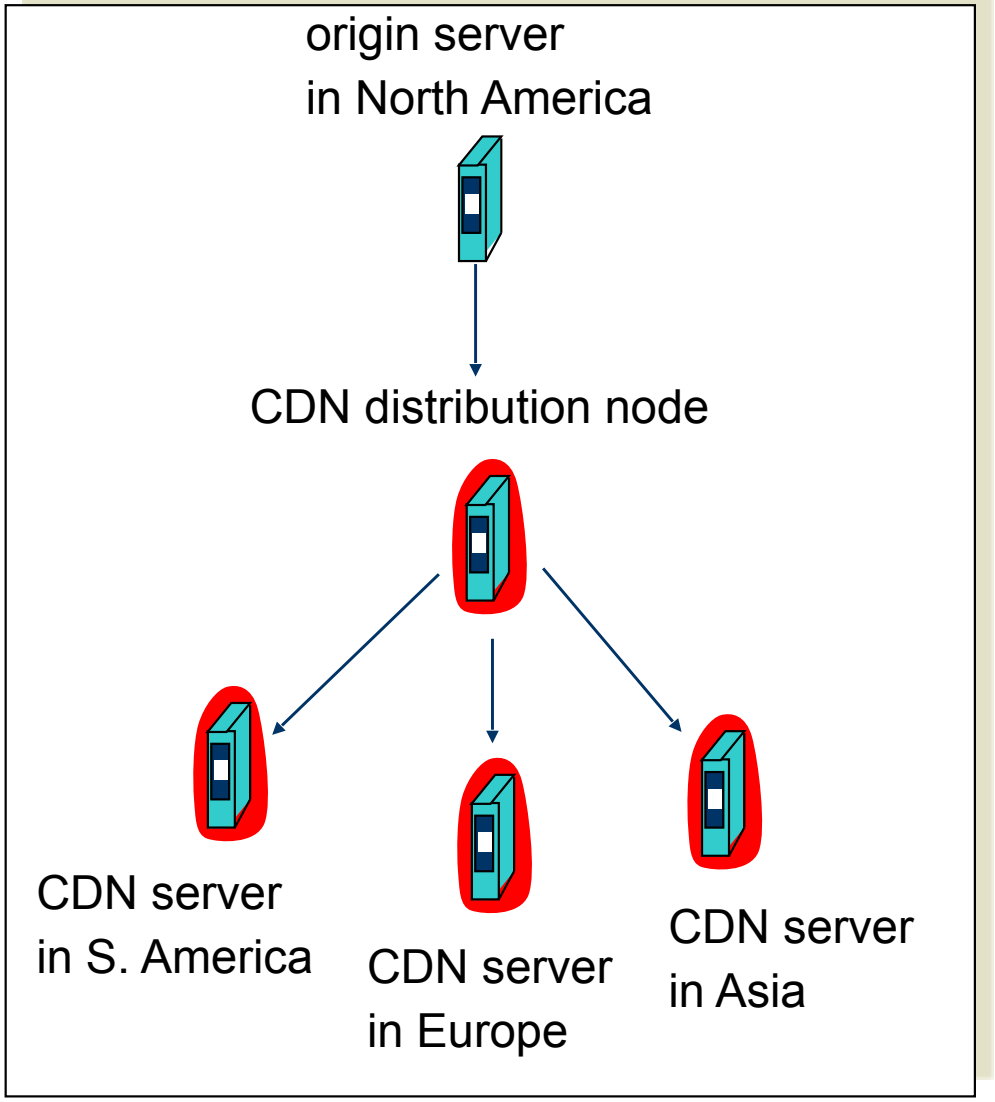


Popular Systems

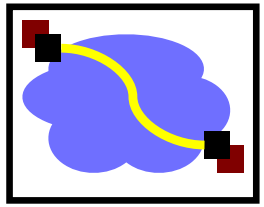
Content Distribution Networks (CDNs)



- The content providers are the CDN customers.
- Content replication
- CDN company installs hundreds of CDN servers throughout Internet
  - Close to users
- CDN replicates its customers' content in CDN servers. When provider updates content, CDN updates servers

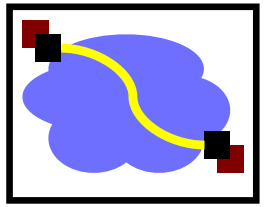


Server Selection



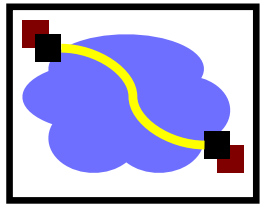
- Which server?
  - Lowest load → to balance load on servers
  - Best performance → to improve client performance
    - Based on Geography? RTT? Throughput? Load?
  - Any alive node → to provide fault tolerance
- How to direct clients to a particular server?
  - As part of routing → anycast, cluster load balancing
    - Not covered ☹
  - As part of application → HTTP redirect
  - As part of naming → DNS

Naming Based



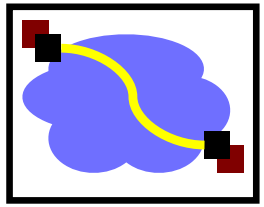
- Client does name lookup for service
- Name server chooses appropriate server address
  - A-record returned is “best” one for the client
- What information can name server base decision on?
  - Server load/location → must be collected
  - Information in the name lookup request
    - Name service client → typically the local name server for client

How Akamai Works



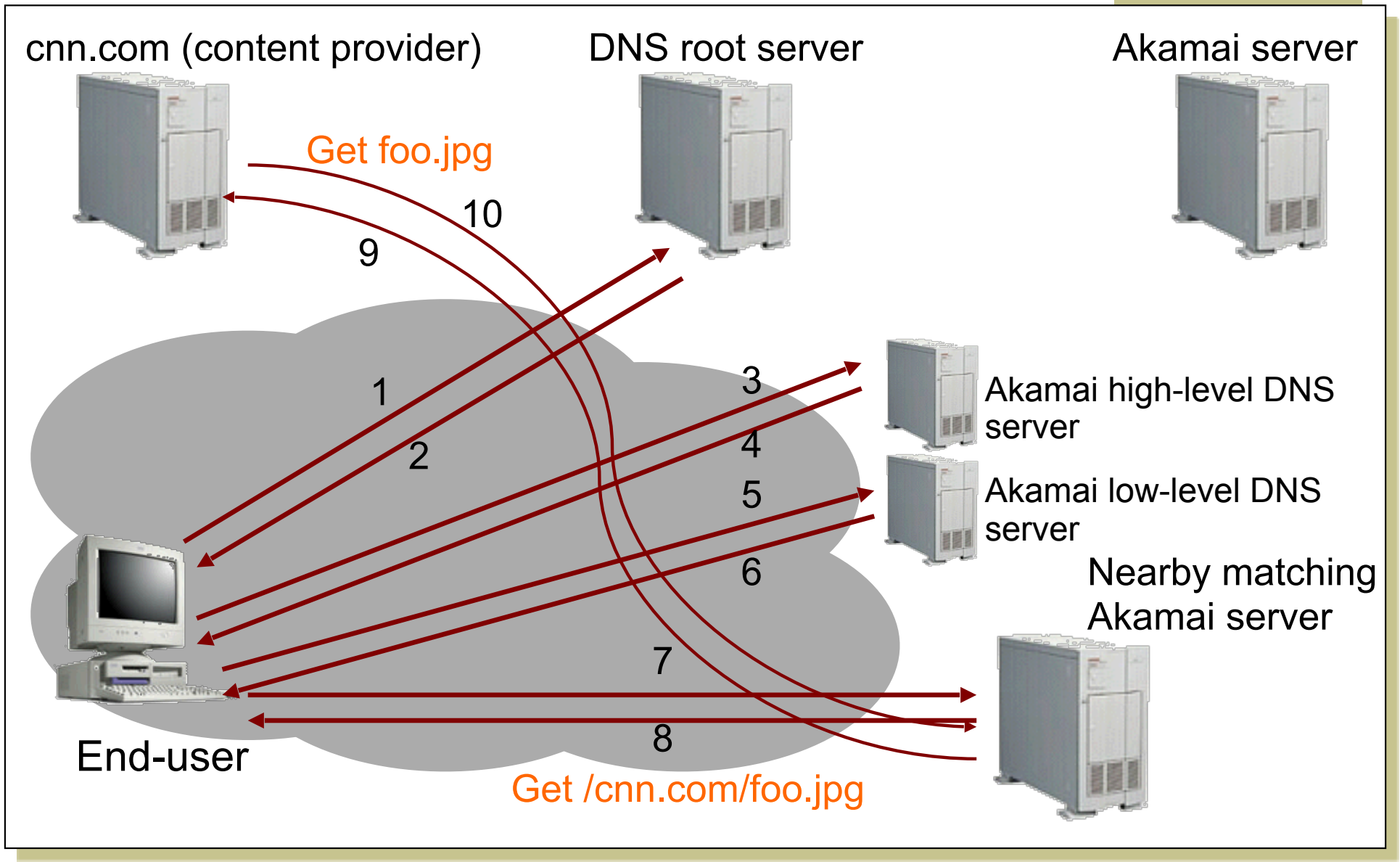
- Clients delegate domain to akamai
  - ibm.com. 172800 IN NS usw2.akam.net.
- CNAME records eventually lead to
  - Something like e2874.x.akamaiedge.net. For IBM
  - Or a1686.q.akamai.net for IKEA….
- Client is forced to resolve eXYZ.x.akamaiedge.net. hostname

How Akamai Works

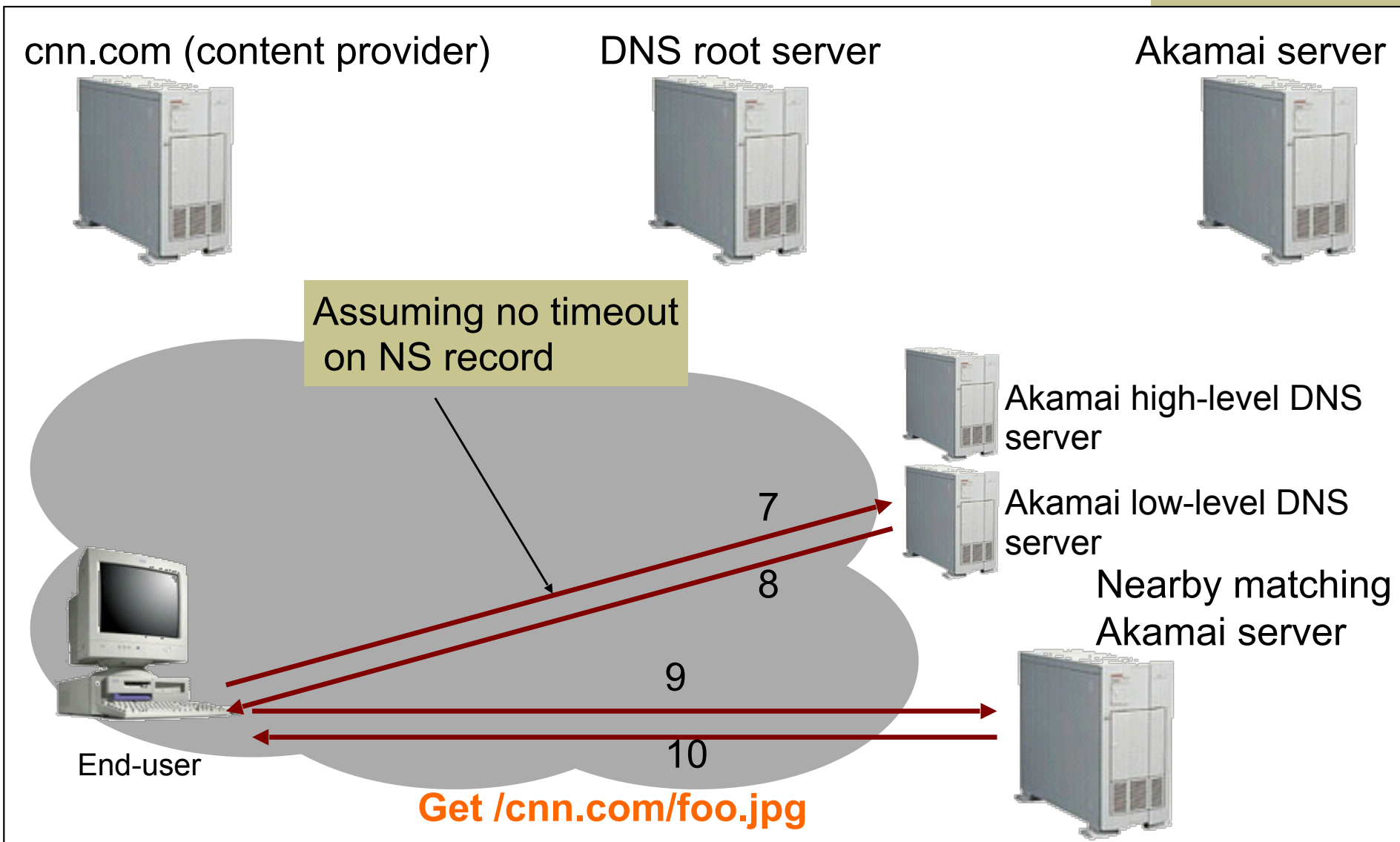


- Root server gives NS record for akamai.net
- Akamai.net name server returns NS record for x.akamaiedge.net
  - Name server chosen to be in region of client's name server
  - TTL is large
- x.akamaiedge.net nameserver chooses server in region
  - Should try to chose server that has file in cache - How to choose?
  - Uses eXYZ name and consistent hashing
  - TTL is small → why?

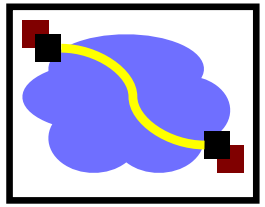
How Akamai Works



Akamai – Subsequent Requests

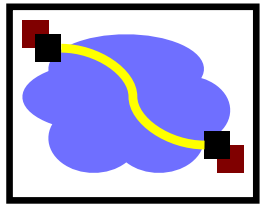


Consistent Hashing Reminder…



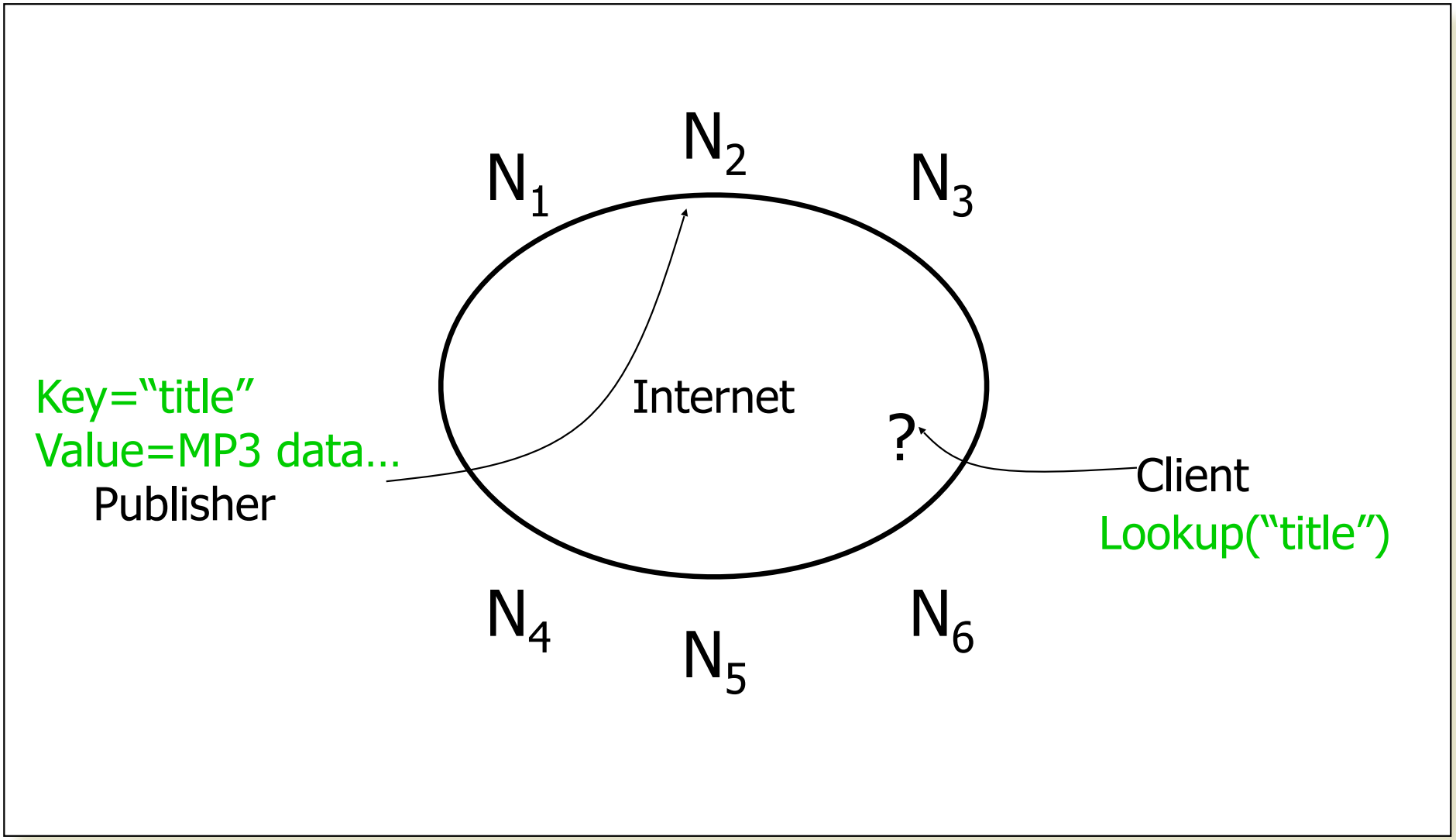
- Finding a nearby server for an object in a CDN uses centralized knowledge.
- Consistent hashing can also be used in a distributed setting
- Consistent Hashing to the rescue.

Peer-to-Peer Networks

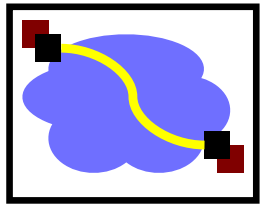


- Typically each member stores/provides access to content
- Basically a replication system for files
  - Always a tradeoff between possible location of files and searching difficulty
  - Peer-to-peer allow files to be anywhere → searching is the challenge
  - Dynamic member list makes it more difficult

The Lookup Problem

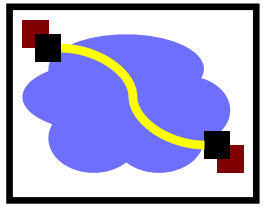


Searching



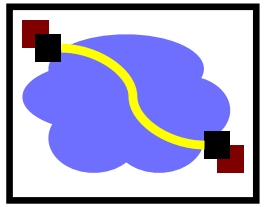
- Needles vs. Haystacks
  - Searching for top 40, or an obscure punk track from 1981 that nobody's heard of?
- Search expressiveness
  - Whole word? Regular expressions? File names? Attributes? Whole-text search?
    - (e.g., p2p gnutella or p2p google?)

Framework



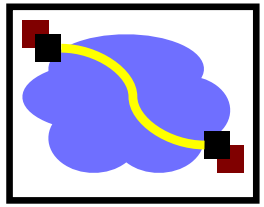
- Common Primitives:
  - **Join**: how to I begin participating?
  - **Publish**: how do I advertise my file?
  - **Search**: how to I find a file?
  - **Fetch**: how to I retrieve a file?

Napster: Overiew



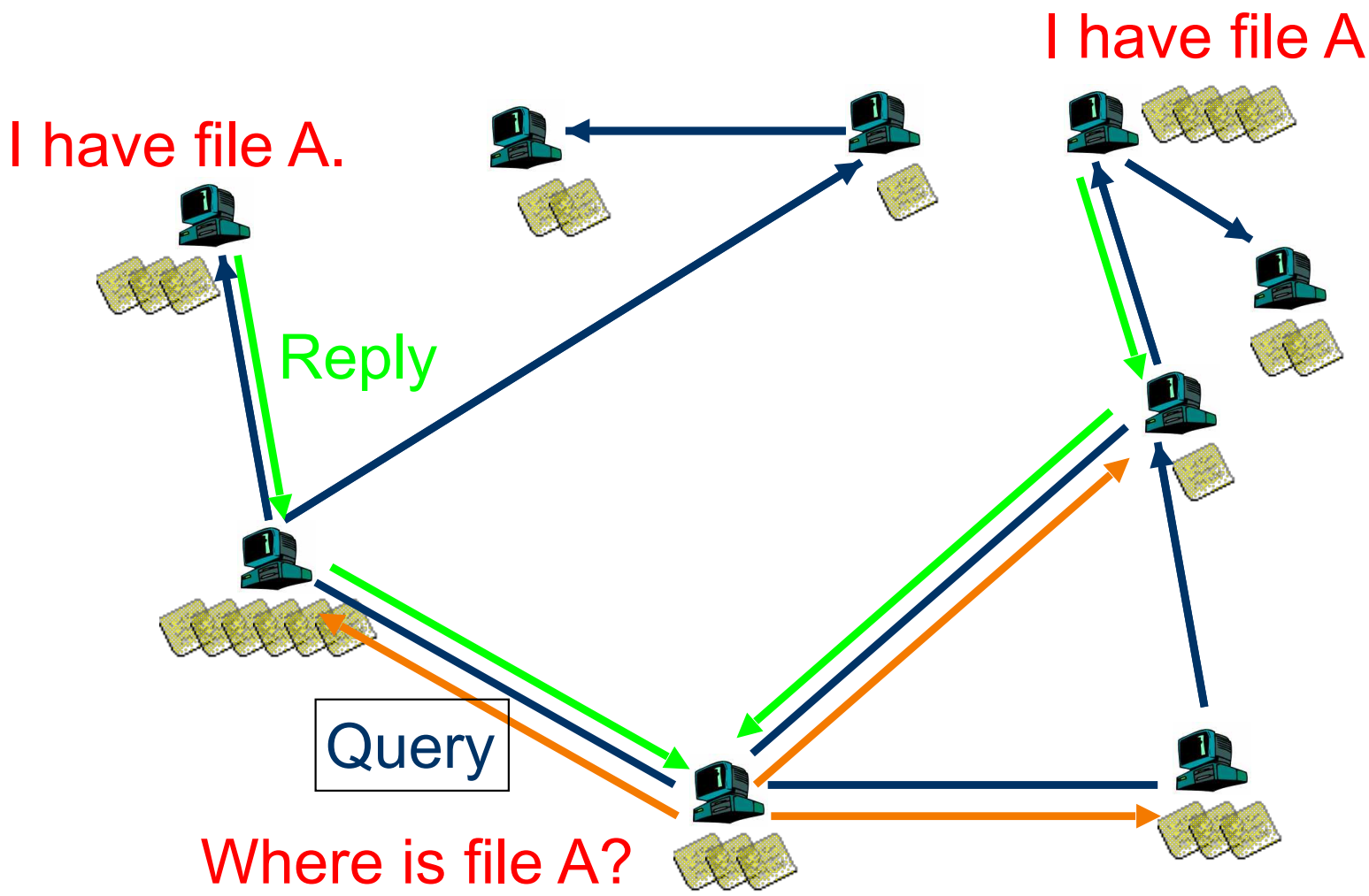
- Centralized Database:
  - **Join:** on startup, client contacts central server
  - **Publish:** reports list of files to central server
  - **Search:** query the server => return someone that stores the requested file
  - **Fetch:** get the file directly from peer

“Old” Gnutella: Overview

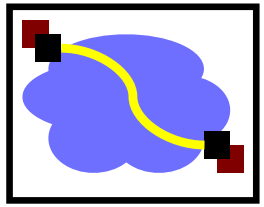


- Query Flooding:
  - **Join**: on startup, client contacts a few other nodes; these become its “neighbors”
  - **Publish**: no need
  - **Search**: ask neighbors, who ask their neighbors, and so on... when/if found, reply to sender.
    - TTL limits propagation
  - **Fetch**: get the file directly from peer

Gnutella: Search

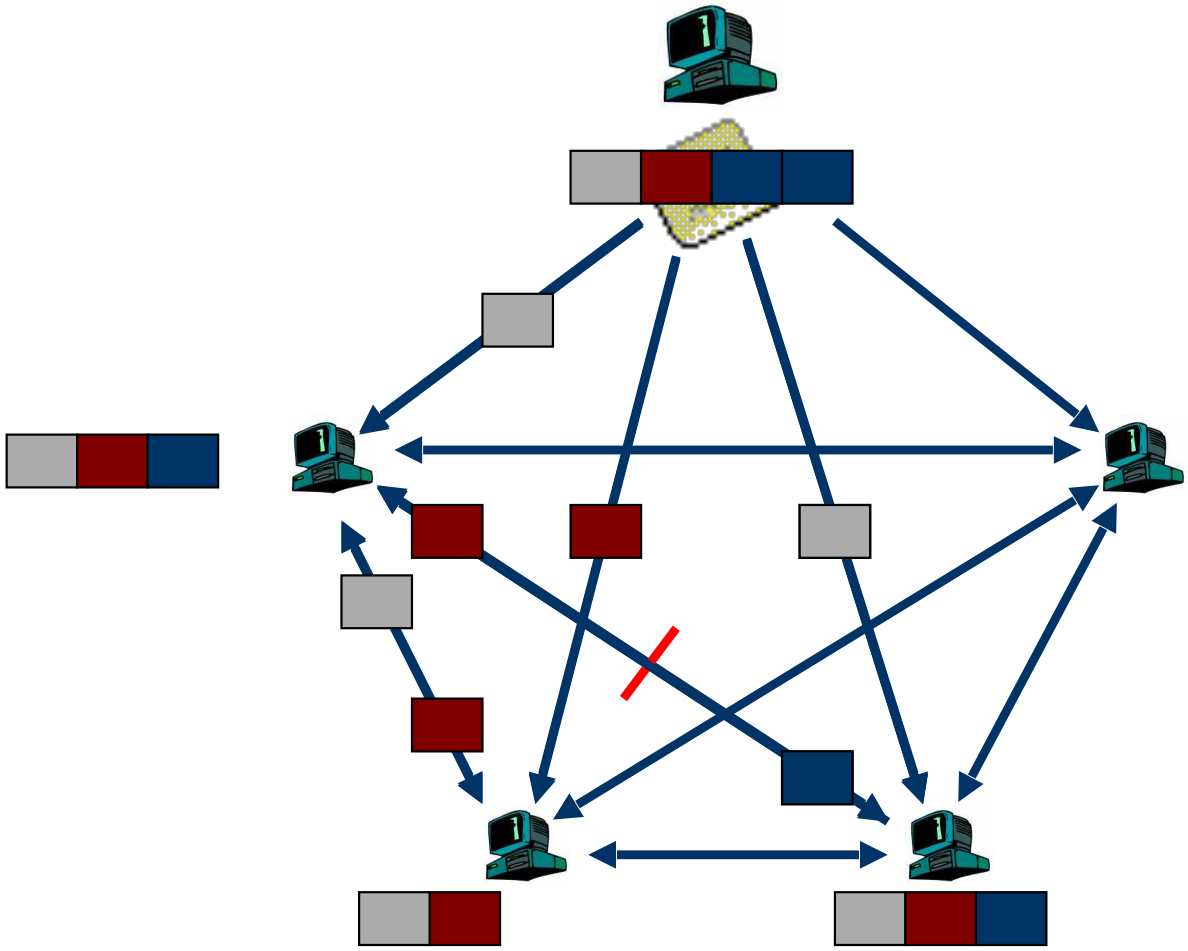


BitTorrent: Overview

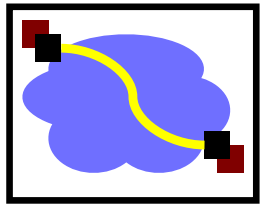


- Swarming:
  - **Join**: contact centralized “tracker” server, get a list of peers.
  - **Publish**: Run a tracker server.
  - **Search**: Out-of-band. E.g., use Google to find a tracker for the file you want.
  - **Fetch**: Download chunks of the file from your peers. Upload chunks you have to them.
- Big differences from Napster:
  - Chunk based downloading
  - “few large files” focus
  - Anti-freeloading mechanisms

BitTorrent: Fetch

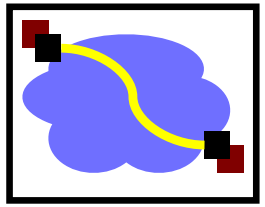


BitTorrent: Sharing Strategy



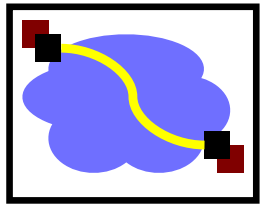
- Employ “Tit-for-tat” sharing strategy
  - A is downloading from some other people
    - A will let the fastest N of those download from him
  - Be optimistic: occasionally let freeloaders download
    - Otherwise no one would ever start!
    - Also allows you to discover better peers to download from when they reciprocate
- Goal: Pareto Efficiency
  - Game Theory: “No change can make anyone better off without making others worse off”
  - Does it work? (not perfectly, but perhaps good enough?)

DHT: Overview (1)

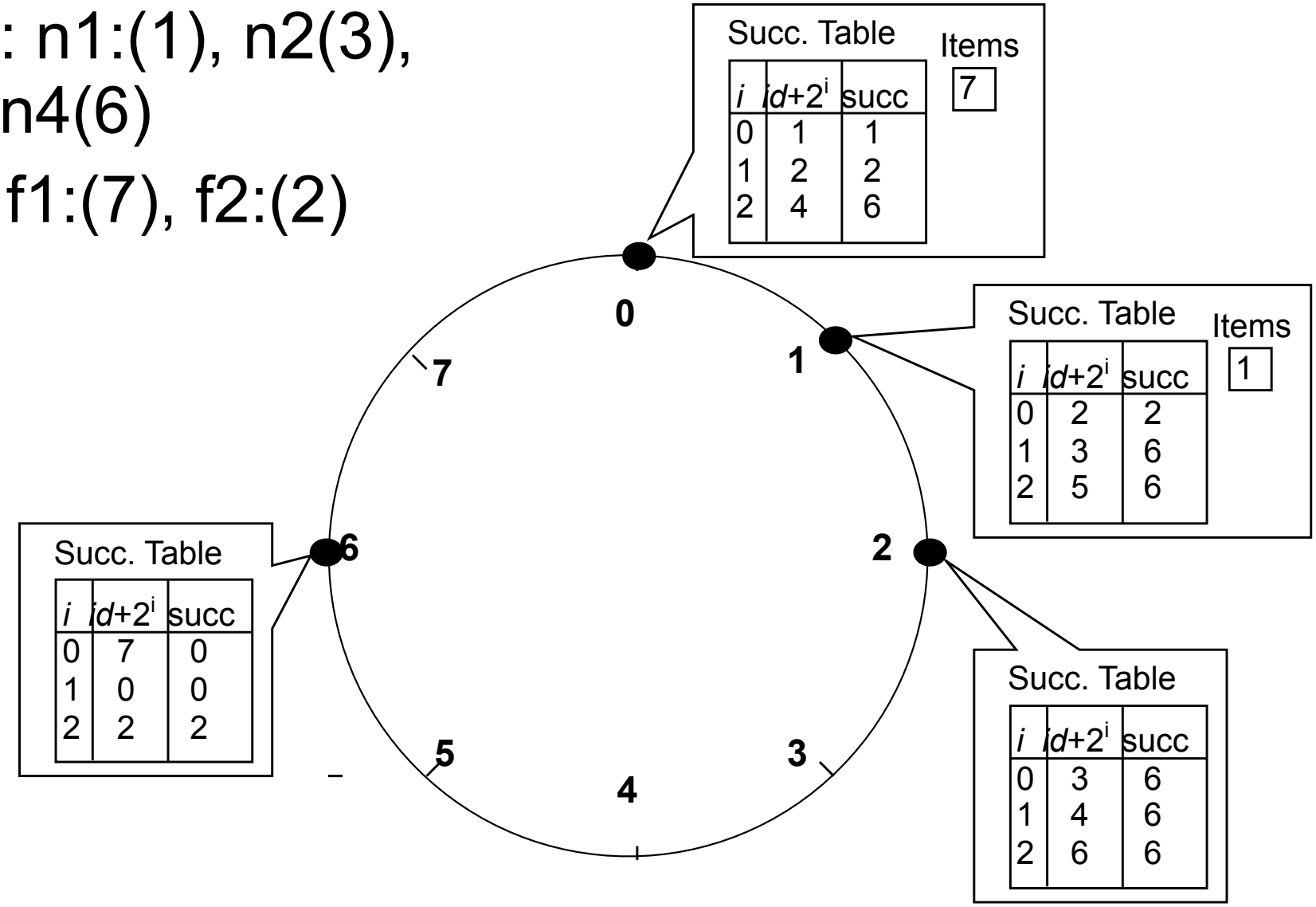


- Goal: make sure that an item (file) identified is always found in a reasonable # of steps
- Abstraction: a distributed hash-table (DHT) data structure
  - insert(id, item);
  - item = query(id);
  - Note: item can be anything: a data object, document, file, pointer to a file…
- Implementation: nodes in system form a distributed data structure
  - Can be Ring, Tree, Hypercube, Skip List, Butterfly Network, ...

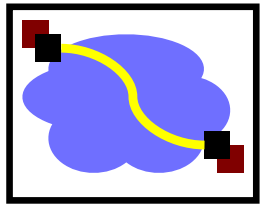
Routing: Chord Examples



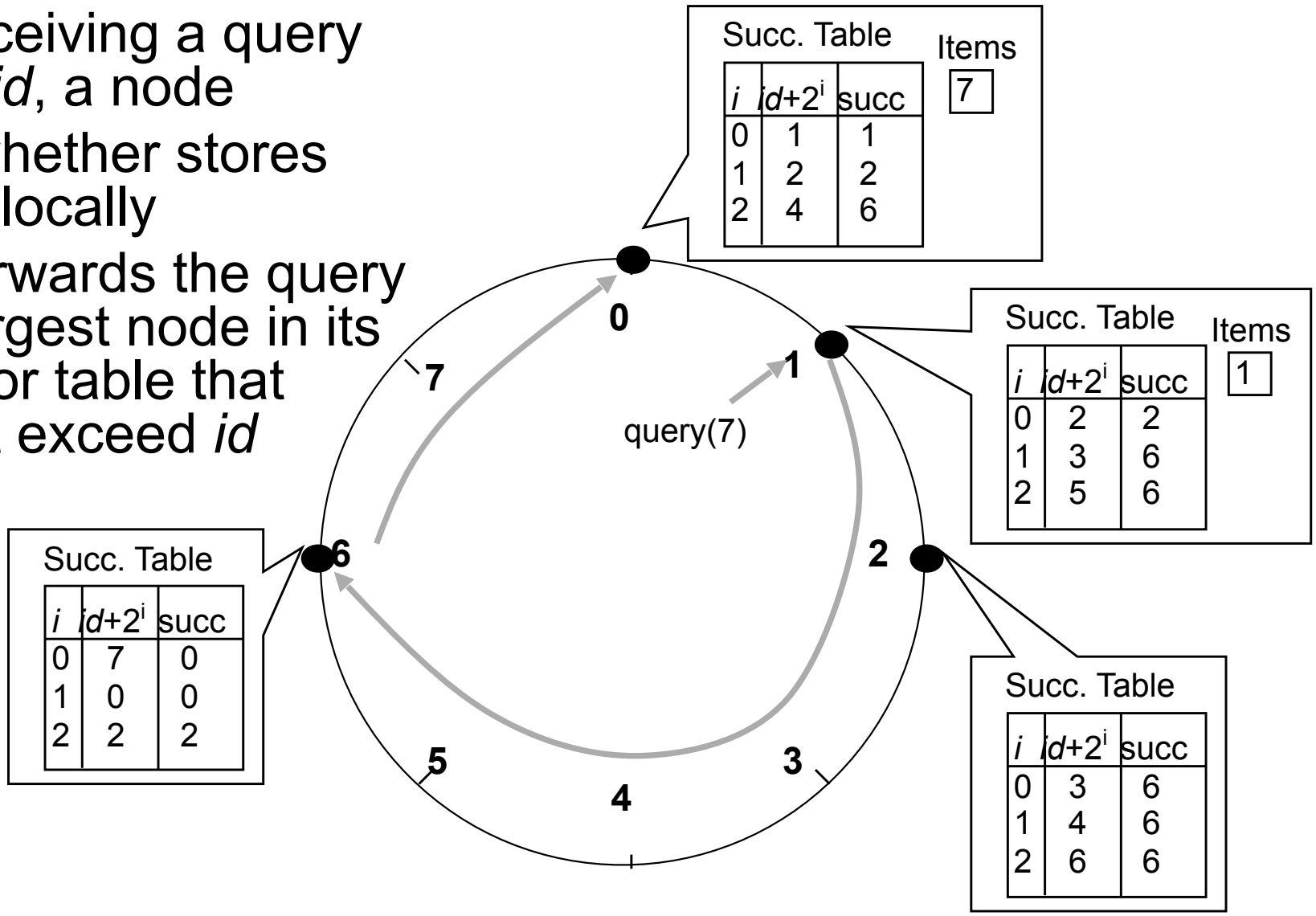
- Nodes: n1:(1), n2(3), n3(0), n4(6)
- Items: f1:(7), f2:(2)



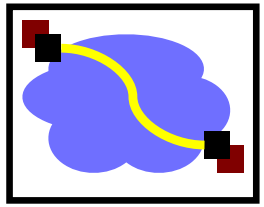
Routing: Query



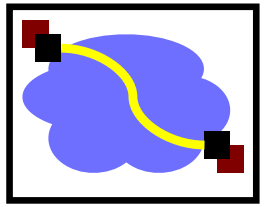
- Upon receiving a query for item *id*, a node
- Check whether stores the item locally
- If not, forwards the query to the largest node in its successor table that does not exceed *id*

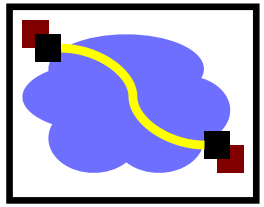


DHT: Discussion



- Pros:
  - Guaranteed Lookup
  - O(log *N*) per node state and search scope
- Cons:
  - Supporting non-exact match search is hard



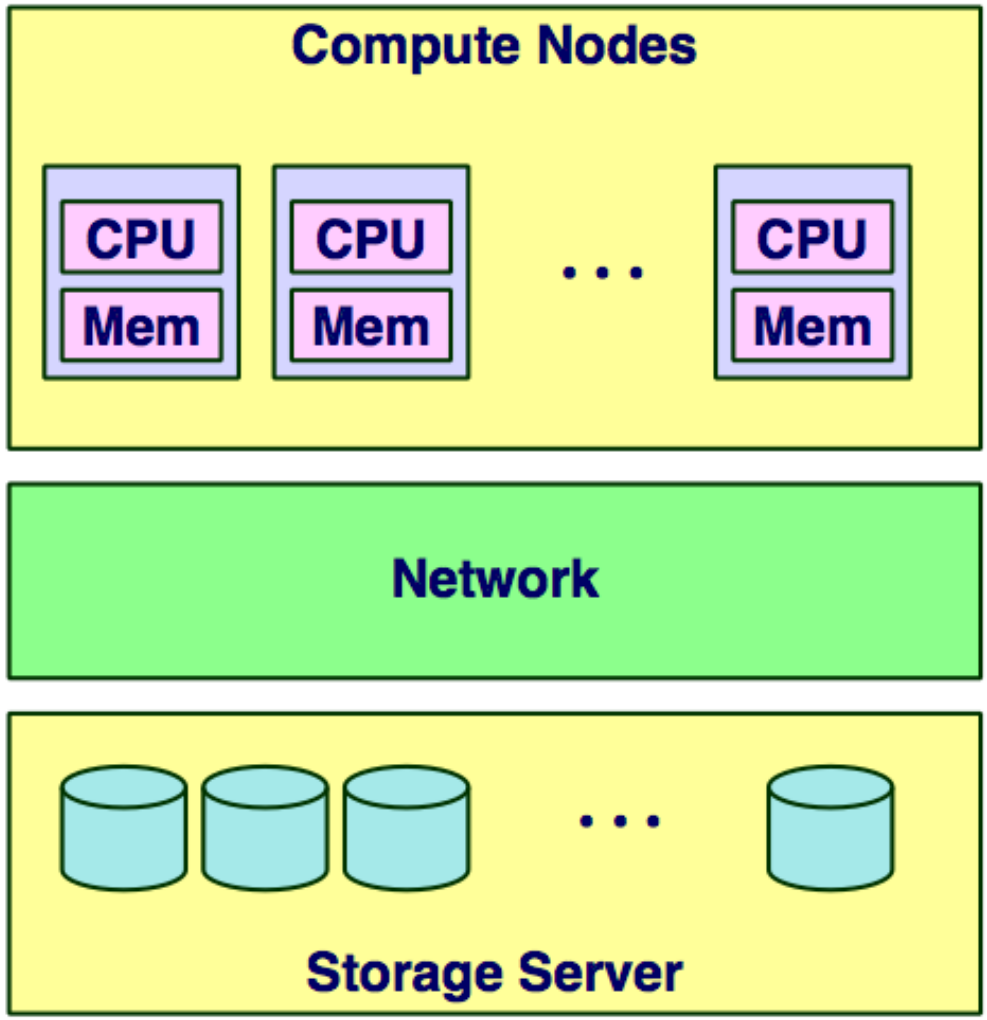


15-440 / 15-640
Lecture 19, November 8th 2016

Data Intensive Computing + MapReduce/Hadoop

- Topics
  - Large-scale computing
    - Traditional high-performance computing (HPC)
    - Cluster computing
  - MapReduce
    - Definition
    - Examples
  - Implementation
  - Alternatives to MapReduce
  - Properties

Typical HPC Machine



Compute Nodes

- High end processor(s)
- Lots of RAM

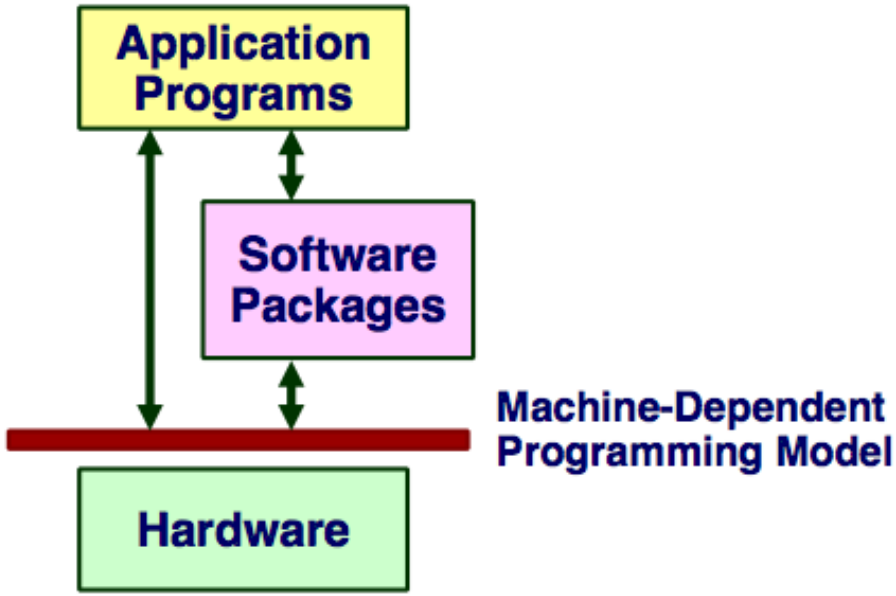
Network

- Specialized
- Very high performance

Storage Server

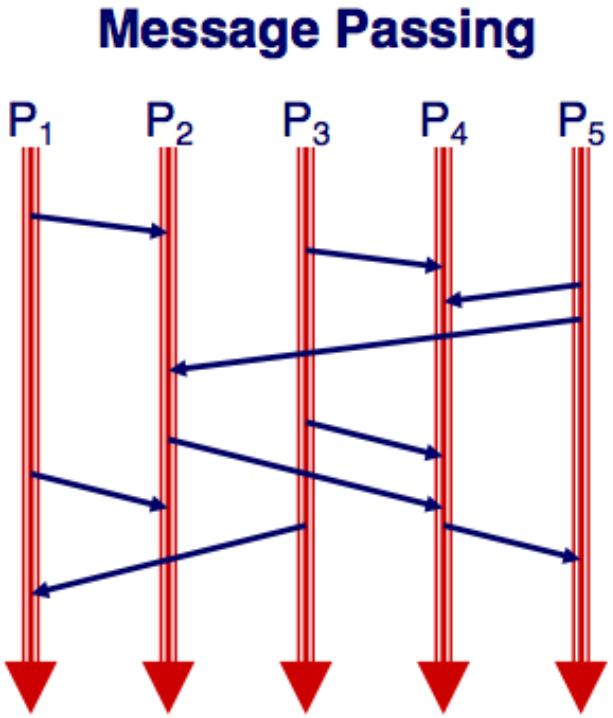
- RAID-based disk array

HPC Programming Model



- **Programs described at very low level**
  - Specify detailed control of processing & communications
- **Rely on small number of software packages**
  - Written by specialists
  - Limits classes of problems & solution methods

Typical HPC Operation



Characteristics

- Long-lived processes
- Make use of spatial locality
- Hold all program data in memory (no disk access)
- High bandwidth communication

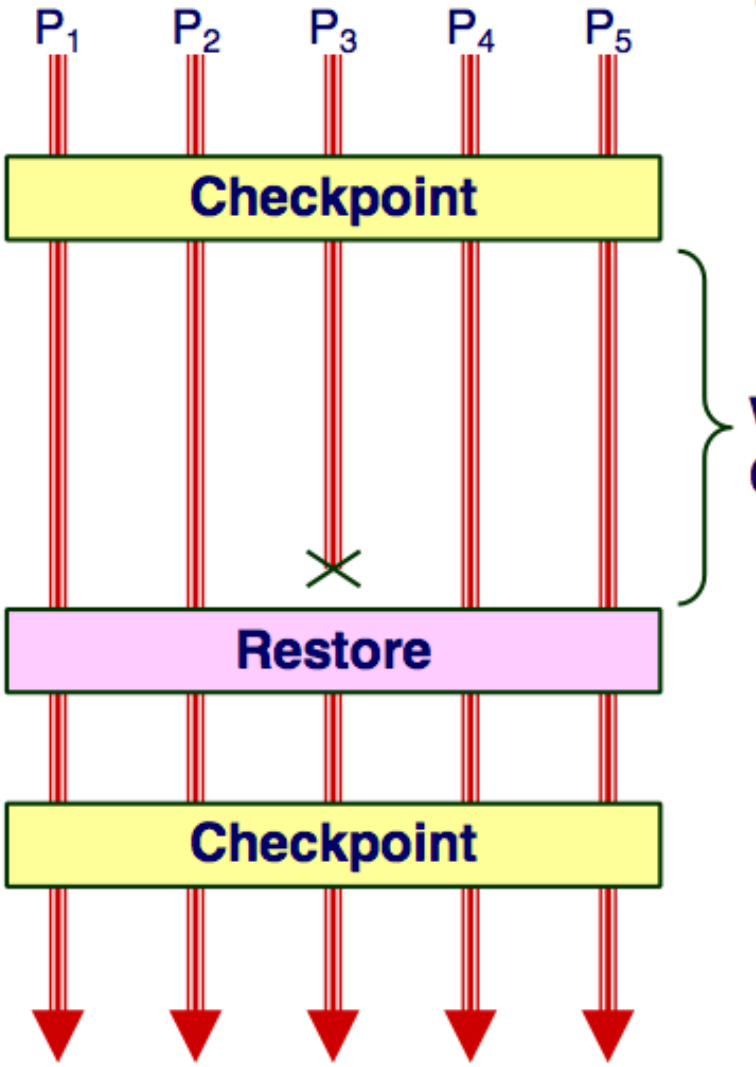
Strengths

- High utilization of resources
- Effective for many scientific applications

Weaknesses

- Requires careful tuning of application to resources
- Intolerant of any variability

HPC Fault Tolerance



Checkpoint

- Periodically store state of all processes
- Significant I/O traffic

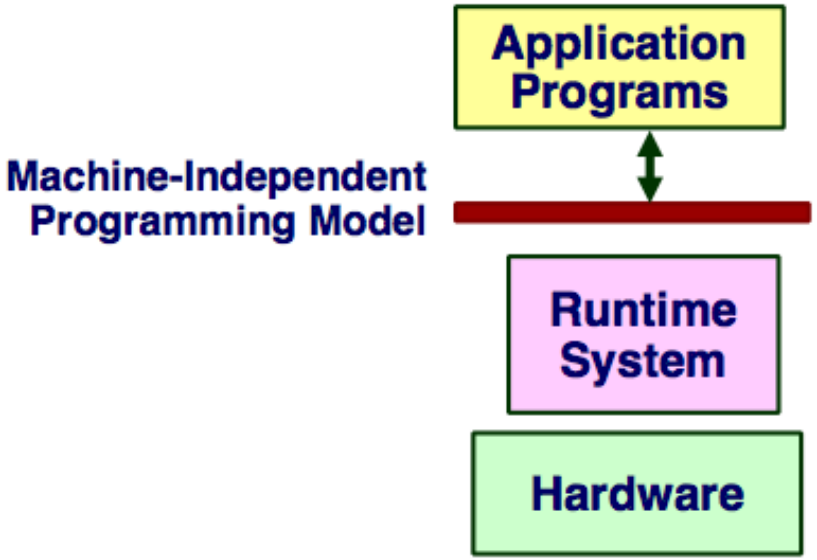
Restore

- When failure occurs
- Reset state to that of last checkpoint
- All intervening computation wasted

Performance Scaling

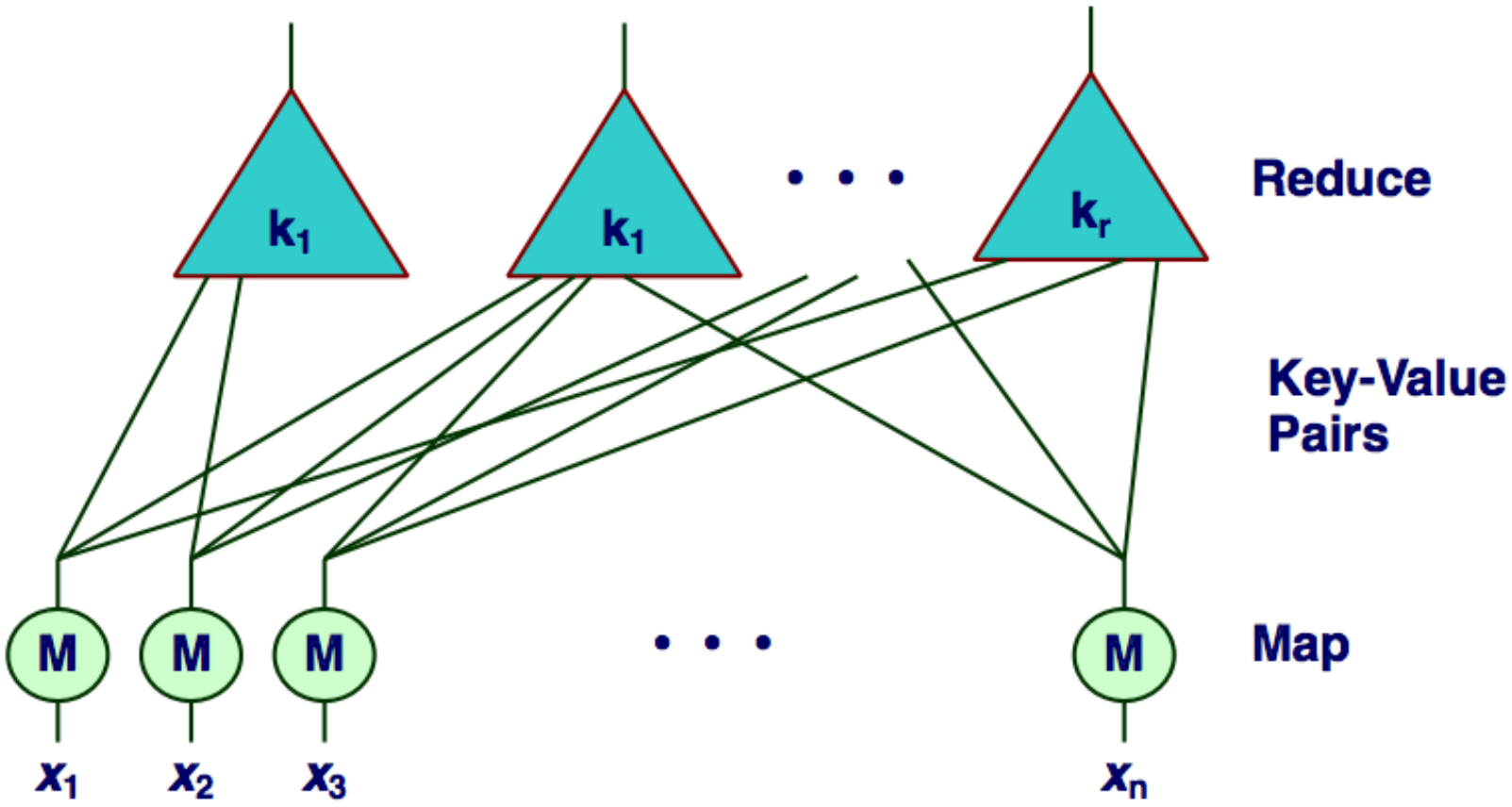
- Very sensitive to number of failing components

Ideal Cluster Programming Model



- Application programs written in terms of high-level operations on data
- Runtime system controls scheduling, load balancing, …

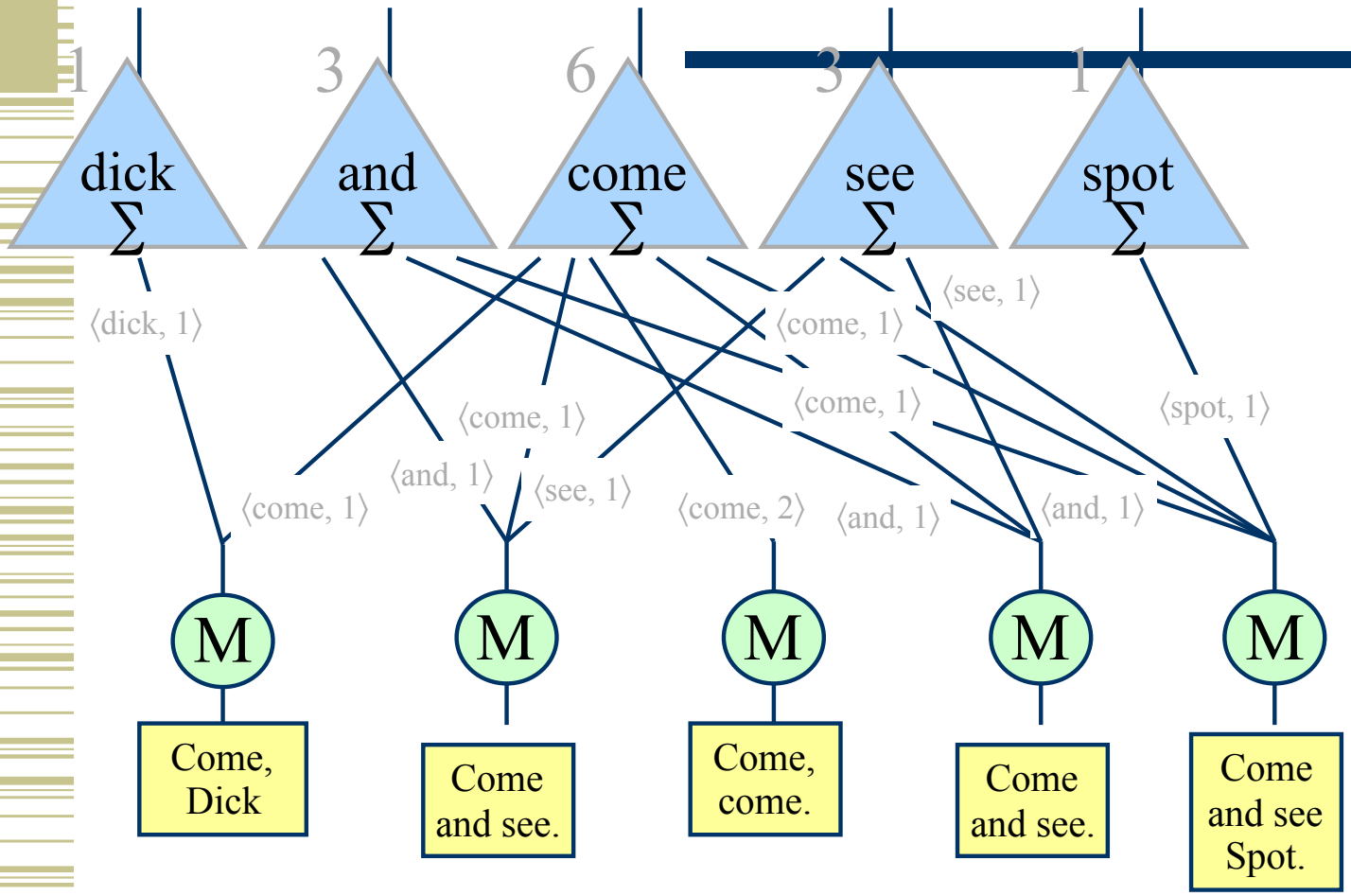
Map/Reduce Programming Model



- **Map computation across many objects**
  - E.g., 10^{10} Internet web pages
- **Aggregate results in many different ways**
- **System deals with issues of resource allocation & reliability**

Dean & Ghemawat: "MapReduce: Simplified Data Processing on Large Clusters", OSDI 2004

MapReduce Example



- Map: generate ⟨word, count⟩ pairs for all words in document
- Reduce: sum word counts across documents

Hadoop MapReduce API

Requirements

- **Programmer must supply Mapper & Reducer classes**

Mapper

- **Steps through file one line at a time**
- **Code generates sequence of <key, value> pairs**
  - Call output.collect(key, value)
- **Default types for keys & values are strings**
  - Lots of low-level machinery to convert to & from other data types
  - But can use anything “writable”

Reducer

- **Given key + iterator that generates sequence of values**
- **Generate one or more <key, value> pairs**
  - Call output.collect(key, value)

Cluster Scalability Advantages

- **Distributed system design principles lead to scalable design**
- **Dynamically scheduled tasks with state held in replicated files**

Provisioning Advantages

- **Can use consumer-grade components**
  - maximizes cost-peformance
- **Can have heterogenous nodes**
  - More efficient technology refresh

Operational Advantages

- **Minimal staffing**
- **No downtime**

MapReduce Implementation

Built on Top of Parallel File System

- Google: GFS, Hadoop: HDFS
- Provides global naming
- Reliability via replication (typically 3 copies)

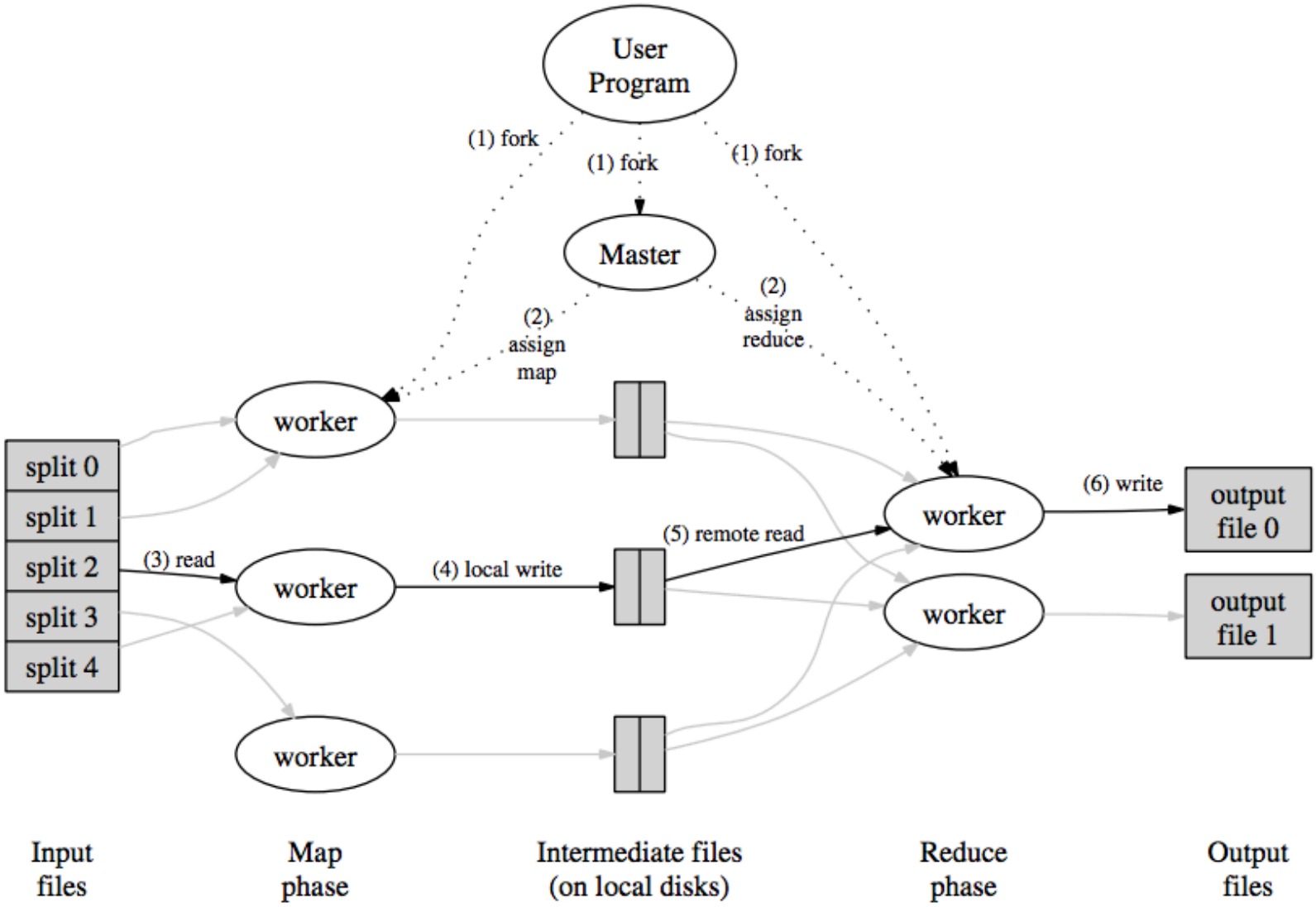
Breaks work into tasks

- Master schedules tasks on workers dynamically
- Typically #tasks >> #processors

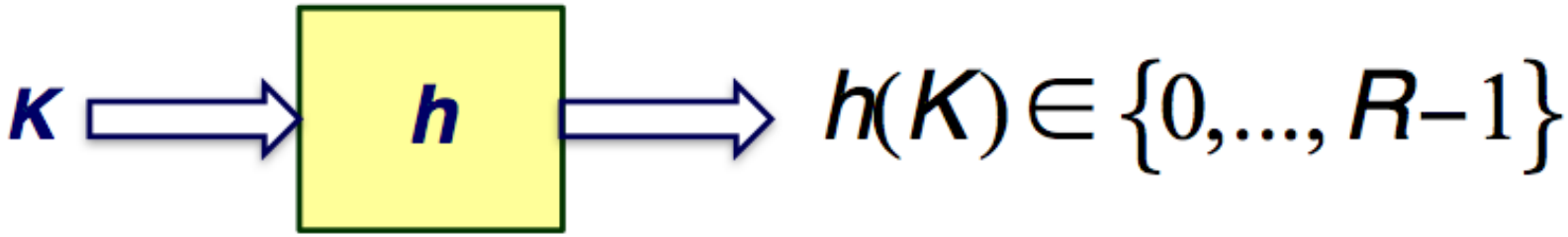
Net Effect

- Input: Set of files in reliable file system
- Output: Set of files in reliable file system

MapReduce Execution (authors view)



Mapping

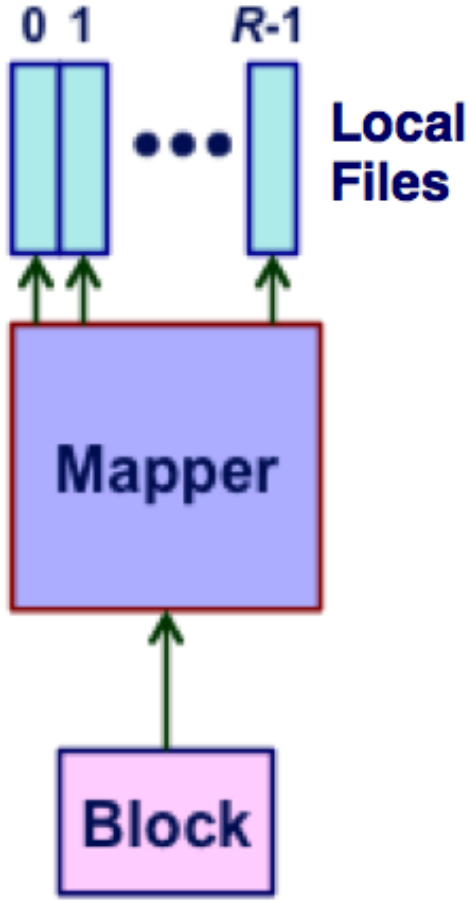


Hash Function *h*

- Maps each key *K* to integer *i* such that $0 \le i < R$

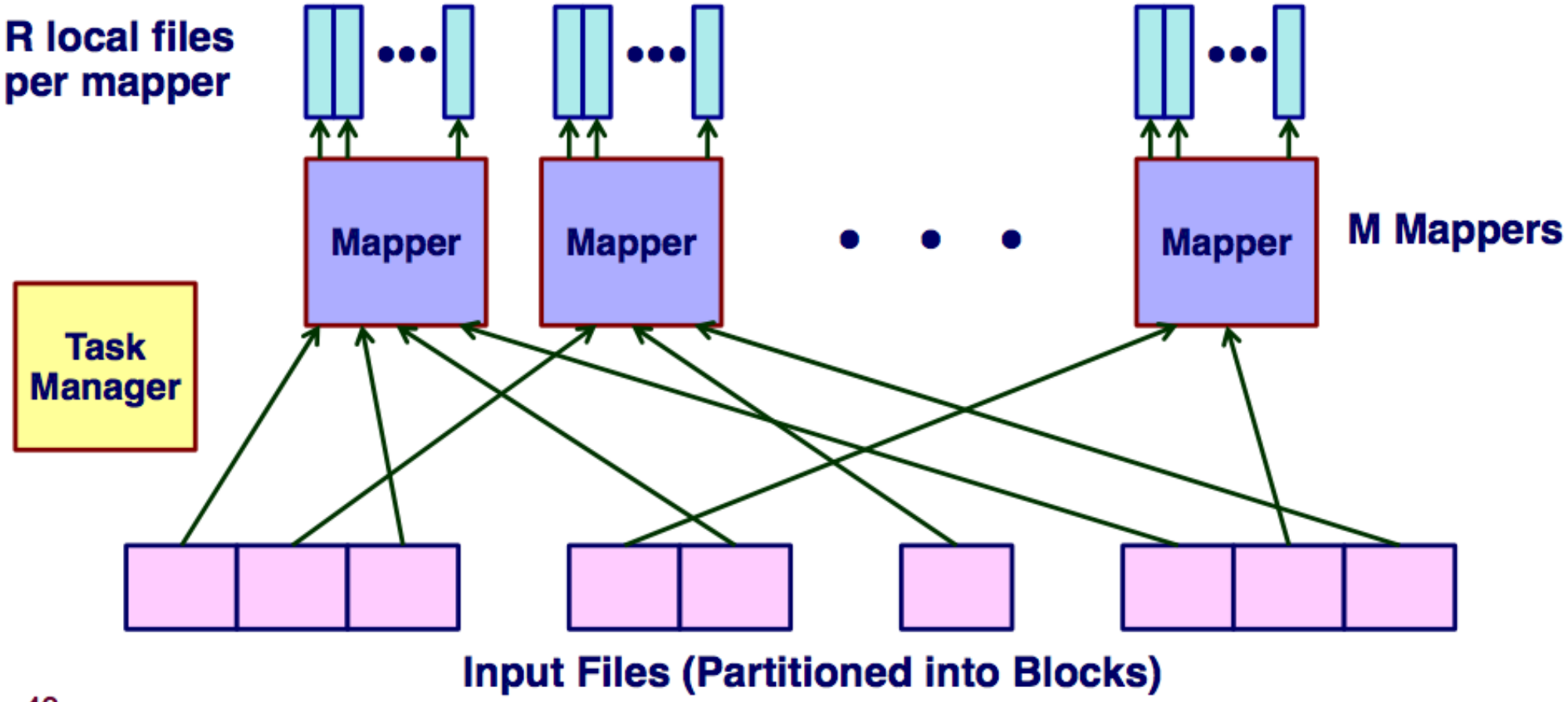
Mapper Operation

- Reads input file blocks
- Generates pairs ⟨*K*, *V*⟩
- Writes to local file *h*(*K*)



Mapping

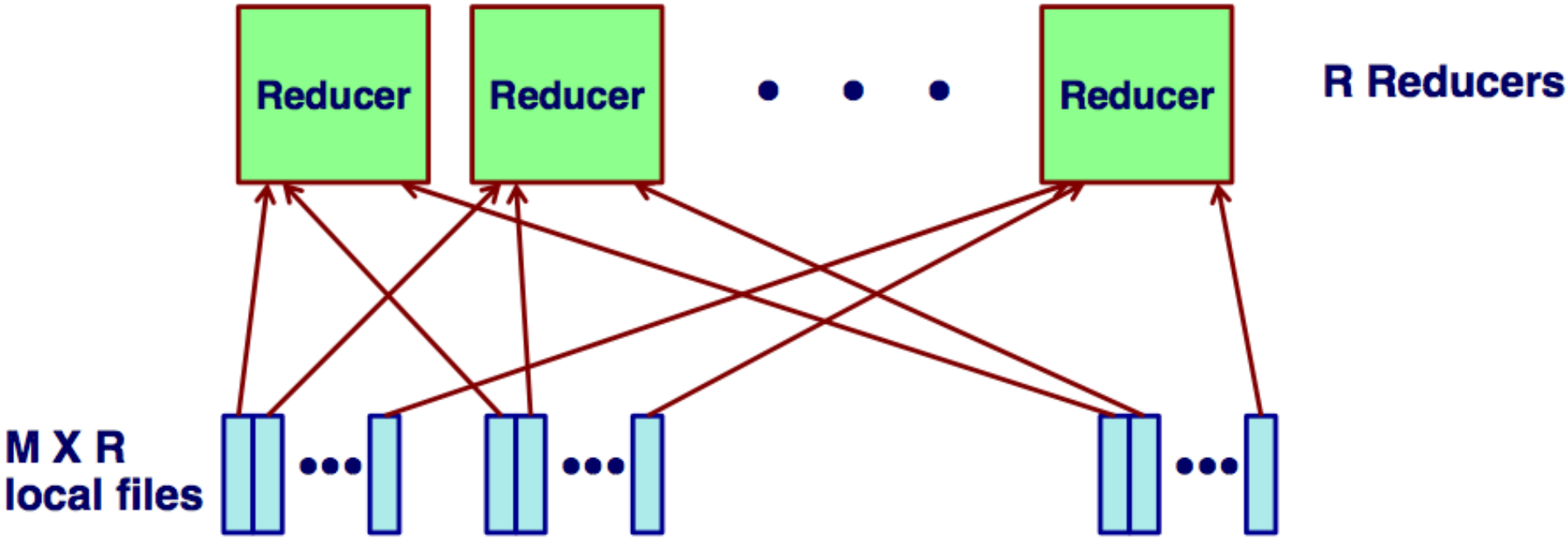
- Dynamically map input file blocks onto mappers
- Each generates key/value pairs from its blocks
- Each writes R files on local file system



Shuffling

Each Reducer:

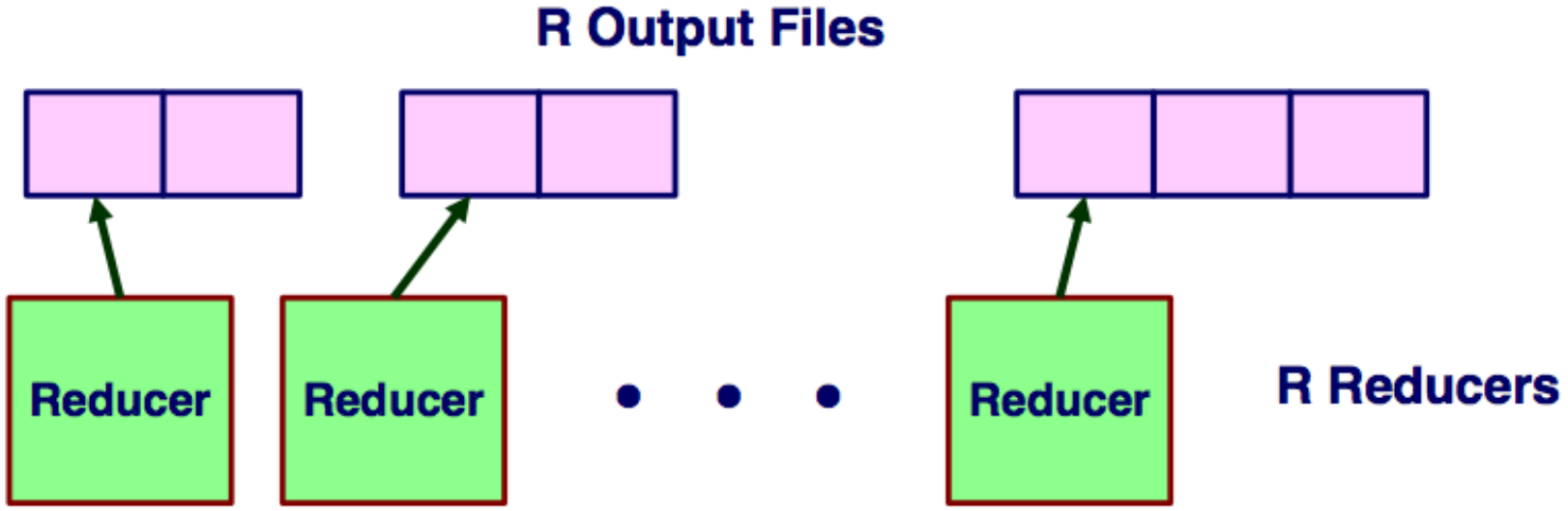
- Handles 1/R of the possible key values
- Fetches its file from each of M mappers
- Sorts all of its entries to group values by keys



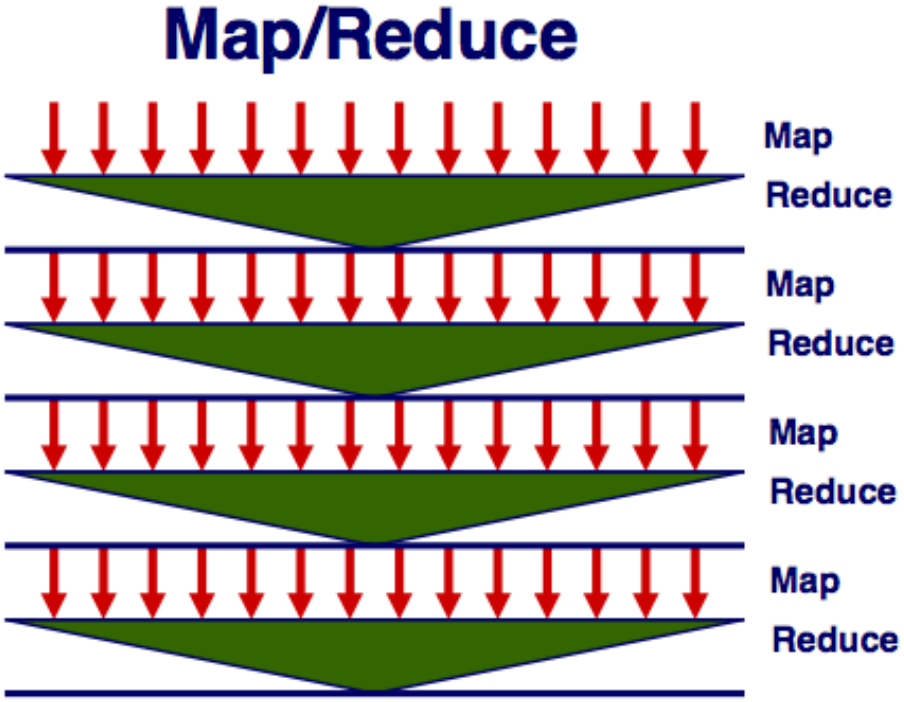
Reducing

Each Reducer:

- Executes reducer function for each key
- Writes output values to parallel file system



Map/Reduce Operation



Characteristics

- Computation broken into many, short-lived tasks
  - Mapping, reducing
- Use disk storage to hold intermediate results

Strengths

- Great flexibility in placement, scheduling, and load balancing
- Can access large data sets

Weaknesses

- Higher overhead
- Lower raw performance

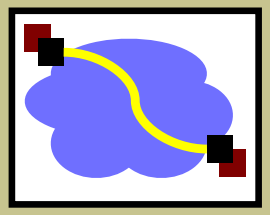
Interesting Features

Fault Tolerance

- **Assume reliable file system**
- **Detect failed worker**
  - Heartbeat mechanism
- **Reschedule failed task**

Stragglers

- **Tasks that take long time to execute**
- **Might be bug, flaky hardware, or poor partitioning**
- **When done with most tasks, reschedule any remaining executing tasks**
  - Keep track of redundant executions
  - Significantly reduces overall run time



15-440 Distributed Systems

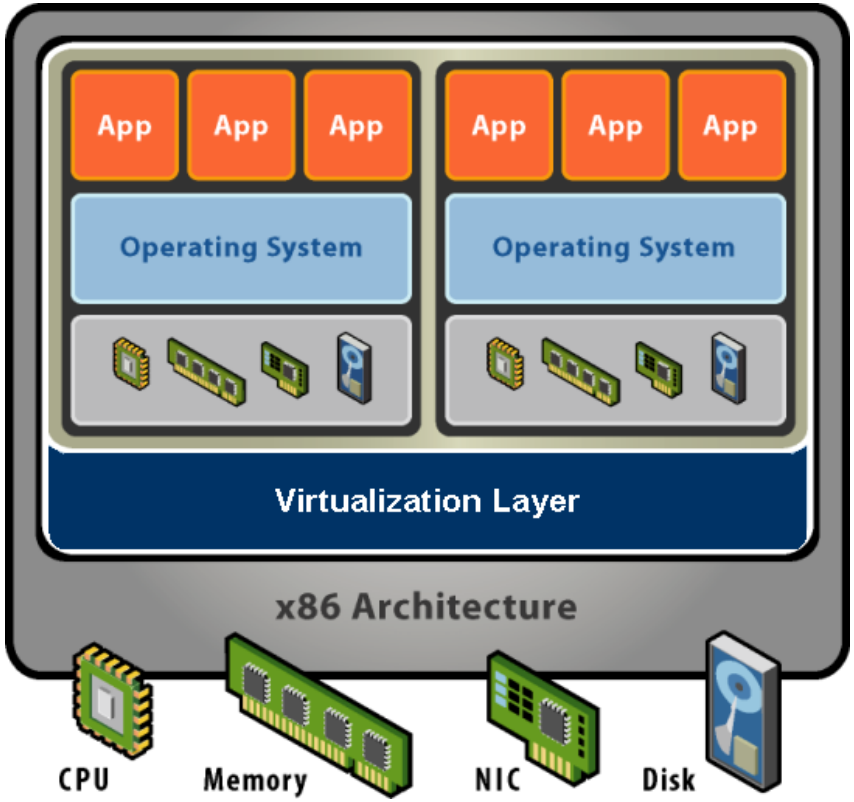
Lecture 20
Virtual Machines
Tuesday Nov 16th, 2015

Some slides based on material from:

Ken Birman @ Cornell, CS6410, Eyal DeLara @ Utoronto, ECE1799

JP Singh, Princeton, COS 318, Alex Snoeren @ UCSD (CSE 120),
OS Concepts (Sliberschatz, Galvin, Gagne 2013)

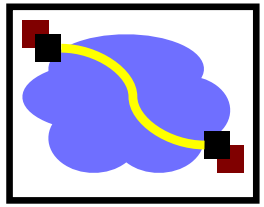
What is a Virtual Machine?



- Software Abstraction
  - Behaves like hardware
  - Encapsulates all OS and application state
- Virtualization Layer
  - Extra level of indirection
  - Decouples hardware, OS
  - Enforces isolation
  - Multiplexes physical hardware across VMs

Adapted from: Eyal DeLara

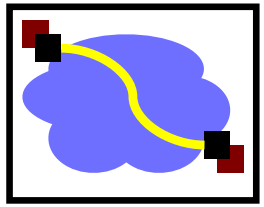
Virtualization Properties, Features



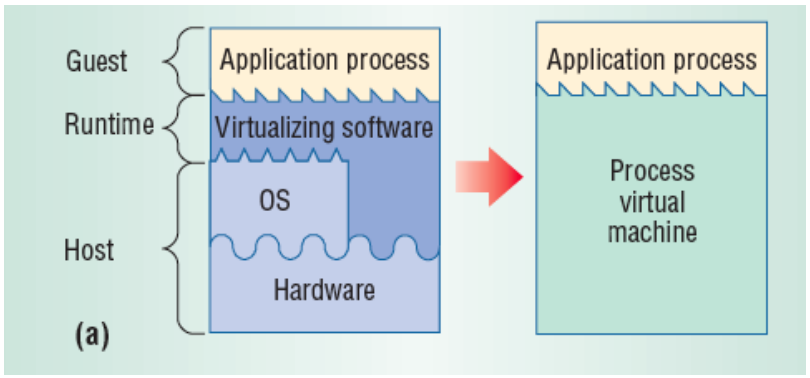
- Isolation
  - Fault isolation
  - Performance isolation (+ software isolation, …)
- Encapsulation
  - Cleanly capture all VM state
  - Enables VM snapshots, clones
- Portability
  - Independent of physical hardware
  - Enables migration of live, running VMs (freeze, suspend,…)
  - Clone VMs easily, make copies
- Interposition
  - Transformations on instructions, memory, I/O
  - Enables transparent resource overcommitment, encryption, compression, replication …

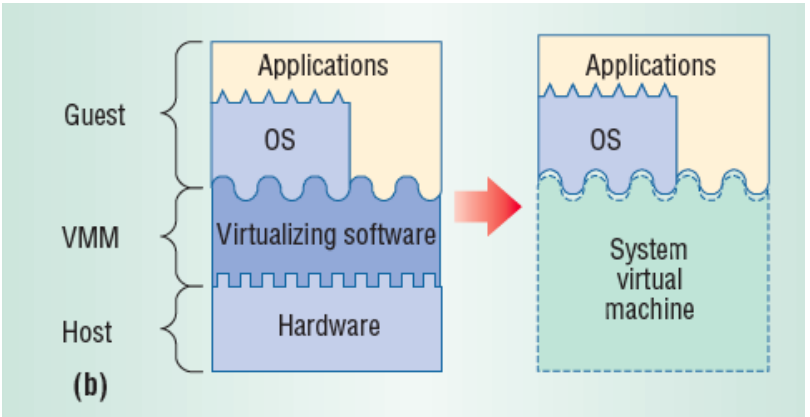
Adapted from: Eyal DeLara

Types of Virtualization



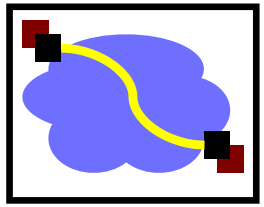
- Process Virtualization (Figure [a])
  - Language-level Java, .NET, Smalltalk
  - OS-level processes, Solaris Zones, BSD Jails, Docker Containers
  - Cross-ISA emulation Apple 68K-PPC-x86
- **System Virtualization** (Figure [b])
  - VMware Workstation, Microsoft VPC, Parallels
  - VMware ESX, Xen, Microsoft Hyper-V



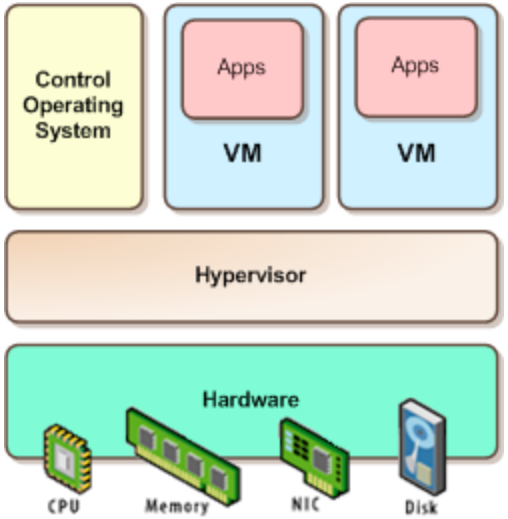


Adapted from: Eyal DeLara

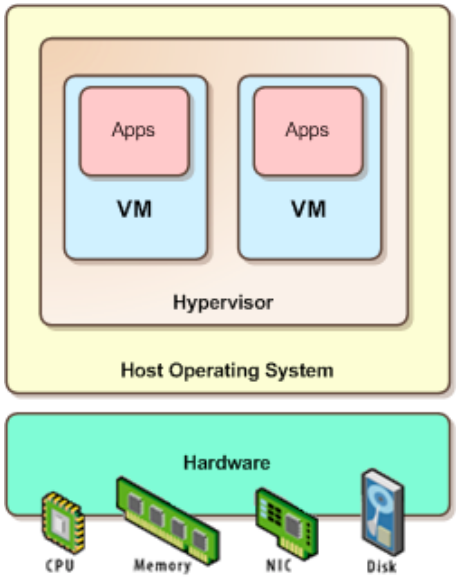
Types of System Virtualization



- Native/Bare metal (Type 1)
  - Higher performance
  - ESX, Xen, HyperV



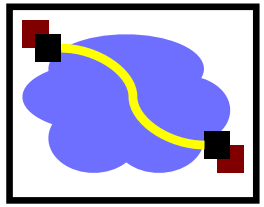
- Hosted (Type 2)
  - Easier to install
  - Leverage host's device drivers
  - VMware Workstation, Parallels



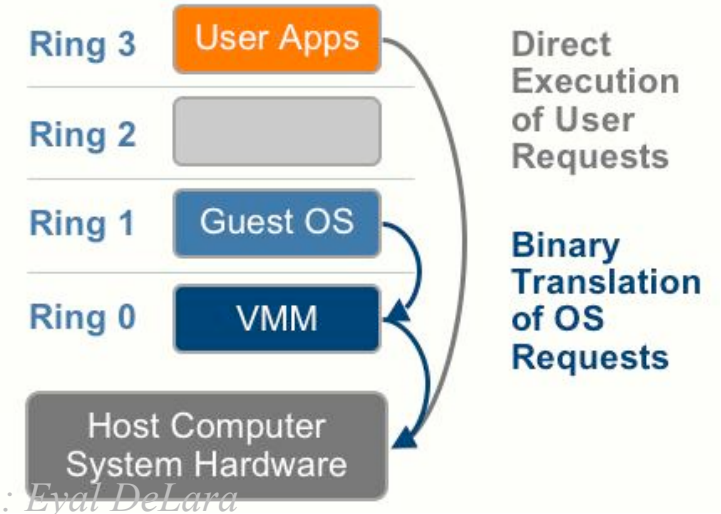
Adapted from: Eyal DeLara

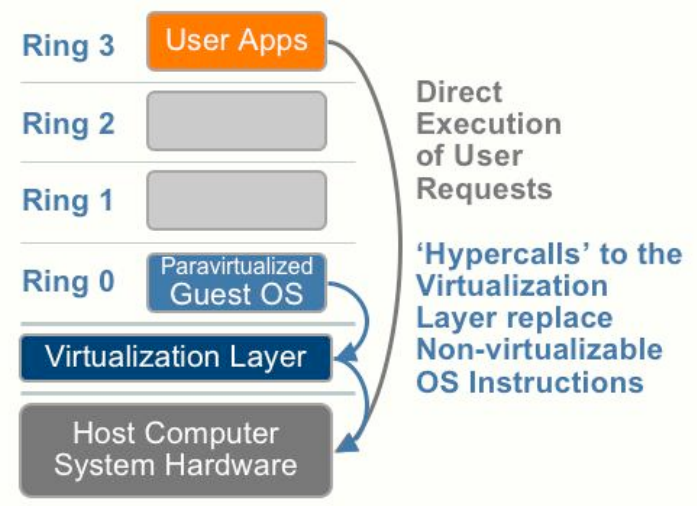
Attribution: http://itechthoughts.wordpress.com/tag/full-virtualization/

Types of Virtualization



- Full virtualization (e.g. VMWare ESX)
  - Unmodified OS, virtualization is transparent to OS
  - VM looks exactly like a physical machine
- Para virtualization (e.g. XEN)
  - OS modified to be virtualized,
  - Better performance at cost of transparency

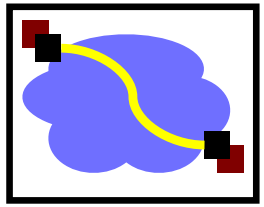




Adapted from: Eyal DeLara

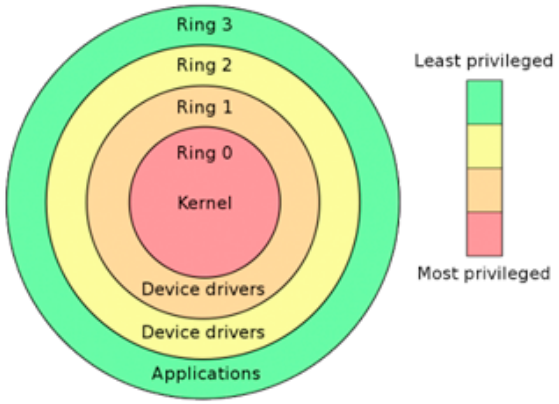
Attribution http://forums.techarena.in/guides-tutorials/1104460.htm

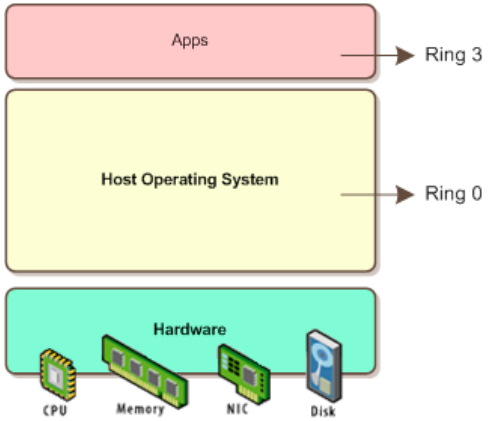
VMM Implementation Goals

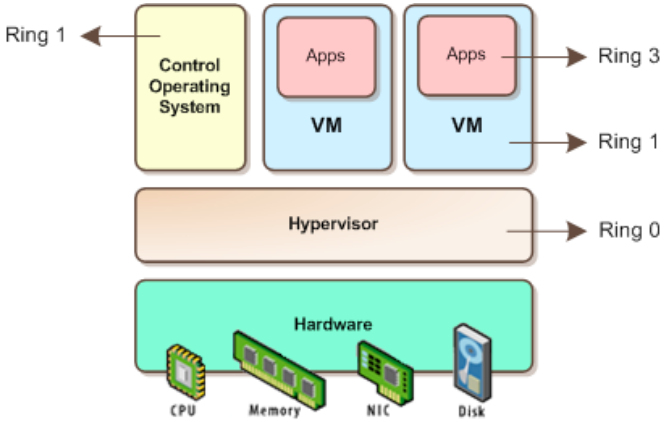


- Should efficiently virtualize the hardware
  - Provide illusion of multiple machines
  - Retain control of the physical machine
- Which subsystems should be virtualized?
  - Processor => Processor Virtualization
  - Memory => Memory Virtualization
  - I/O Devices => I/O virtualization

Processor Virtualization







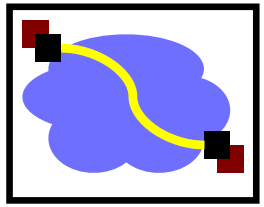
An architecture is classically/strictly virtualizable if all its sensitive instructions (those that violate safety and encapsulation) are a subset of the privileged instructions.

- **all instructions either trap or execute identically**
- **instructions that access privileged state trap**

Attribution: http://itechthoughts.wordpress.com/tag/full-virtualization/

Adapted from: Eyal DeLara

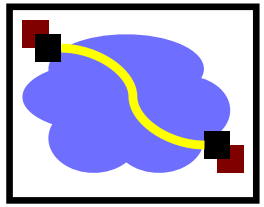
System Call Example



Process	Operating System	VMM
1.System call: Trap to OS		
		2. Process trapped: call OS trap handler (at reduced privilege)
	3. OS trap handler: Decode trap and execute syscall; When done: issue return-frrom-trap	
		4. OS tried to return from trap; do real return-from-trap
5. Resume execution (@PC after trap)		

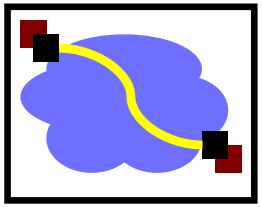
- *Run guest operating system deprivileged*
- *All privileged instructions trap into VMM*
- *VMM emulates instructions against virtual state*
  e.g. disable virtual interrupts, not physical interrupts
- *Resume direct execution from next guest instruction*

Virtualizing Memory

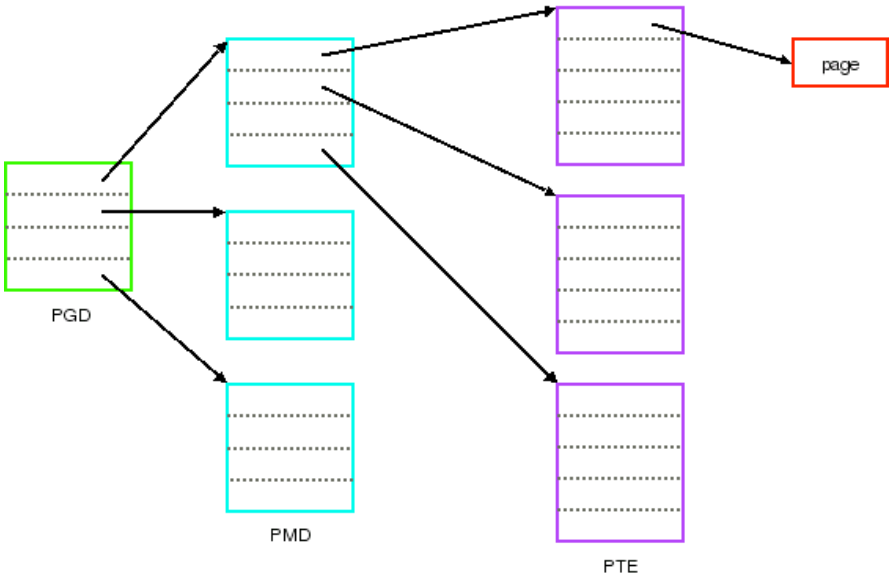


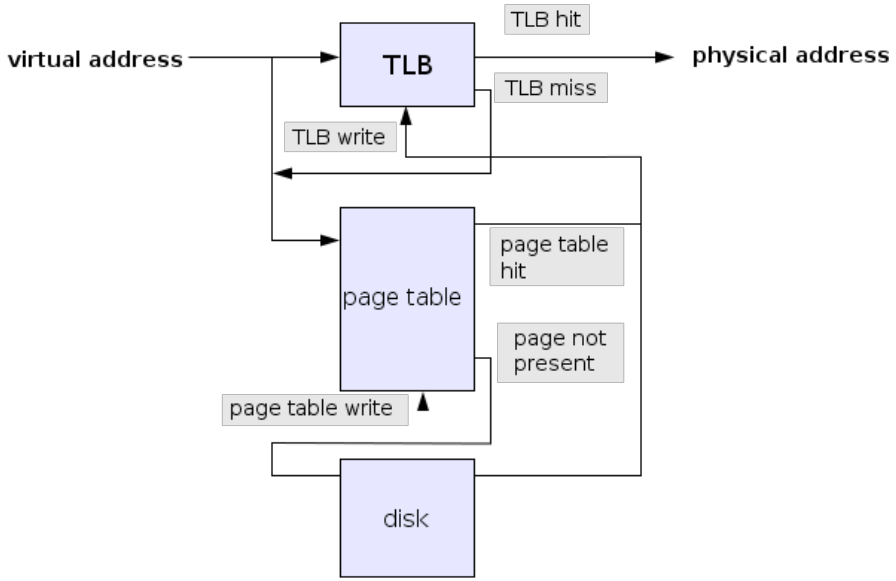
- OS assumes that it has full control over memory
  - Management: Assumes it owns it all
  - Mapping: Assumes it can map any Virtual-> Physical
- However, VMM partitions memory among VMs
  - VMM needs to assign hardware pages to VMs
  - VMM needs to control mapping for isolation
    - Cannot allow OS to map any Virtual => hardware page
- Hardware-managed TLBs make this difficult
  - On TLB misses, the hardware walks page tables in mem
  - VMM needs to control access by OS to page tables

x86 Memory Management Primer

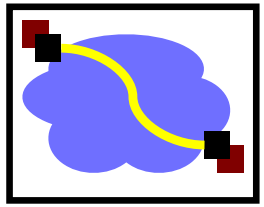


- The processor operates with virtual addresses
- Physical memory operates with physical addresses
- x86 includes a hardware translation lookaside bufer (TLB)
  - Maps virtual to physical page addresses
- x86 handles TLB misses in HW
  - HW walks the page tables => Inserts virtual to physical mapping



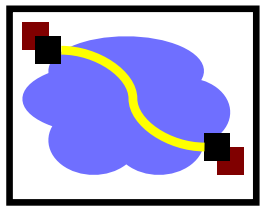


Shadow Page Tables

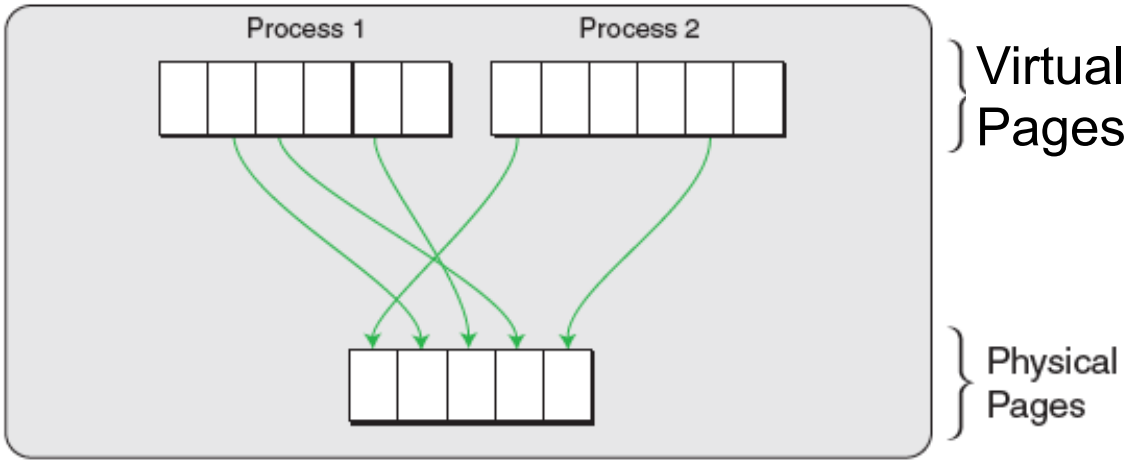


- Three abstractions of memory
  - **Machine:** actual hardware memory (e.g. 2GB of DRAM)
  - **Physical:** abstraction of hardware memory, OS managed
    - E.g. VMM allocates 512 MB to a VM, the OS thinks the computer has 512 MB of contiguous physical memory
    - (Underlying machine memory may be discontiguous)
  - **Virtual:** virtual address space
    - Standard 2^32 address space
- In each VM, OS creates and manages page tables for its virtual address spaces without modification
  - But these page tables **are not used** by the MMU

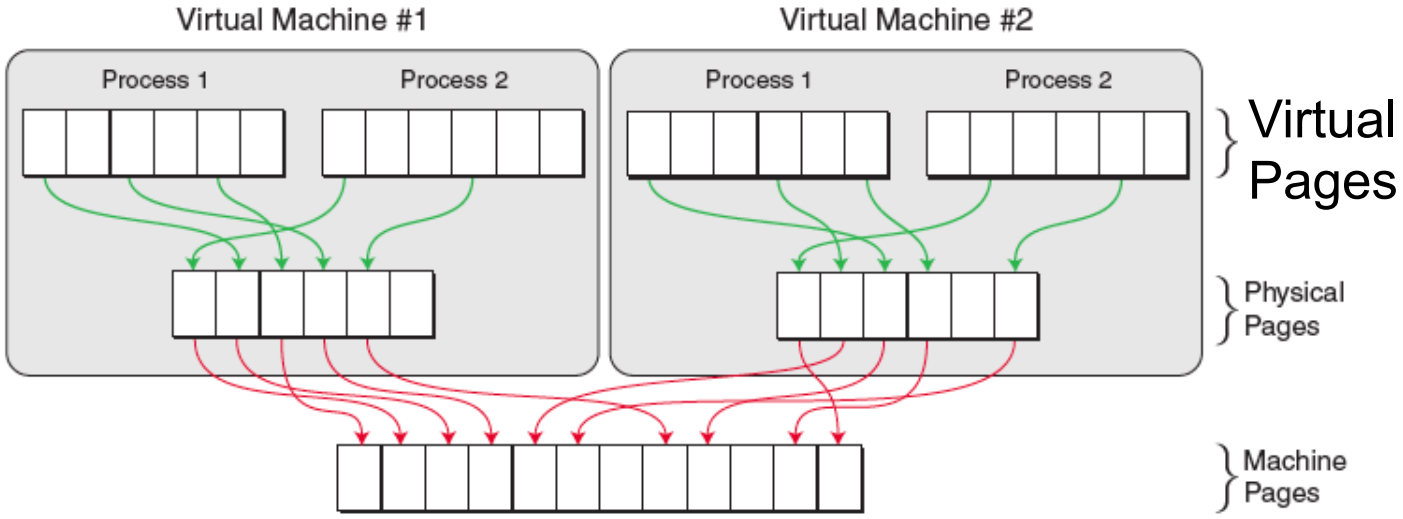
Three Abstractions of memory



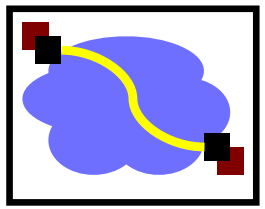
- Native



- Virtualized

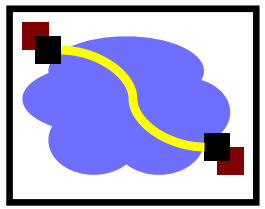


Memory Management / Allocation

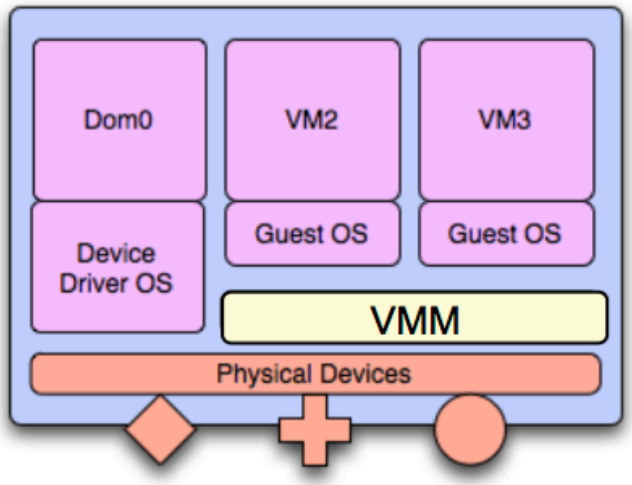


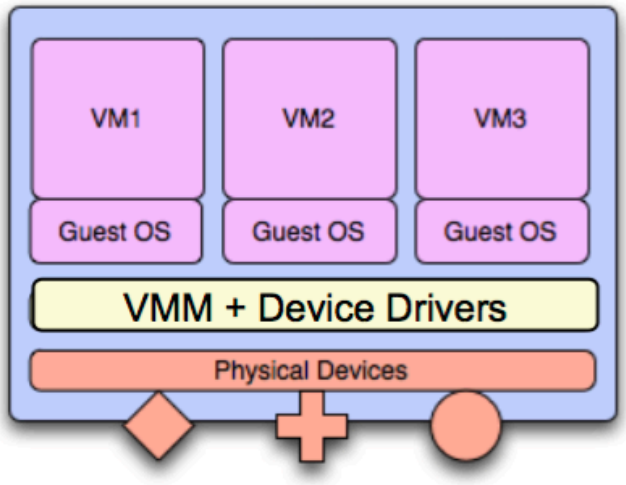
- VMMs tend to have simple memory management
  - Static policy: VM gets 8GB at start
    - Dynamic adjustment is hard since OS cannot handle
    - No swapping to disk
- More sophistication: Overcommit with balooning
  - Baloon driver runs inside OS => consume hardware pages
  - Baloon grows or shrinks (gives back mem to other VMs)
- Even more sophistication: memory de-duplication
  - Identify pages that are shared across VMs!

I/O Virtualization

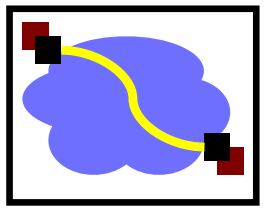


- Challenge: Lots of I/O devices
  - Problem: Writing device drivers for all I/O devices in the VMM layer is not a feasible option
- Insight: Device driver already written for popular Operating Systems
  - Solution: Present *virtual* I/O devices to *guest* VMs and channel I/O requests to a trusted *host* VM (popular OS)



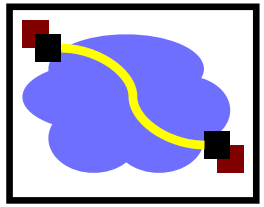


VM Storage Management

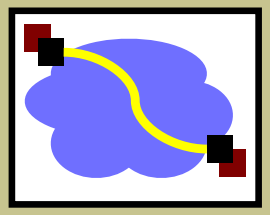


- VMM provides both boot disk + other storage
  - Type 1 VMM – storage guest root disks and config information within file system provided by VMM as a **disk image**
  - Type 2 VMM – store as files in the host file system/OS
- Example of supported operations:
  - Duplicate file (clone) -> create new guest
  - Move file to another system -> move guest
  - Convert formats: **Physical-to-virtual** (**P-to-V**) and **Virtual-to-physical** (**V-to-P**)
- VMM also usually provides access to network attached storage (just networking) => live migration

Summary

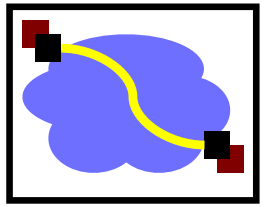


- Introduction to Virtualization: why do it?
- Different types of Virtualization: Container based, Language Based, Type I/II, Paravirtualized.
- VMM techniques to virtualize the CPU, memory, I/O devices, etc
- Example: XEN, open source VMM



Reliability

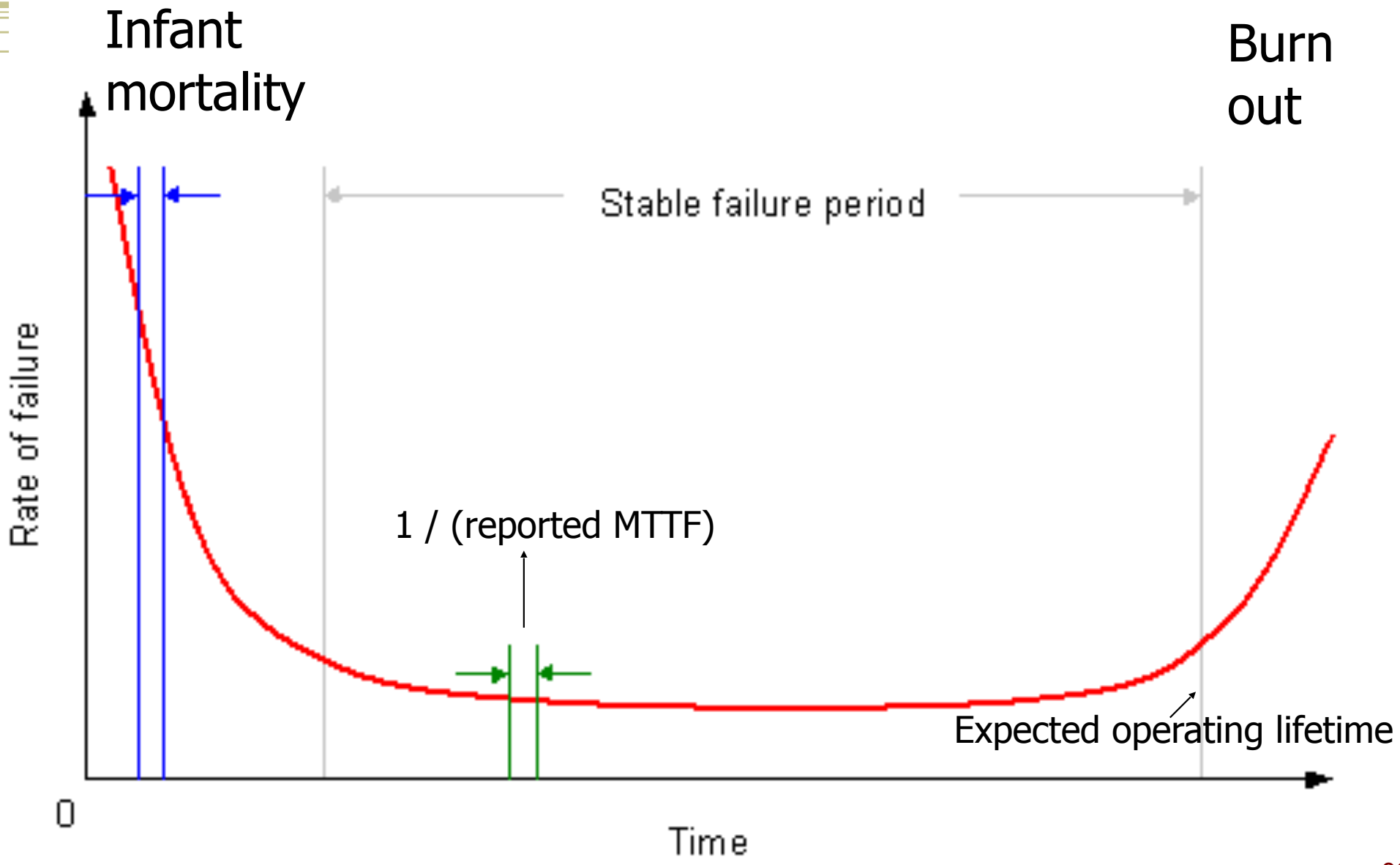
Measuring Availability



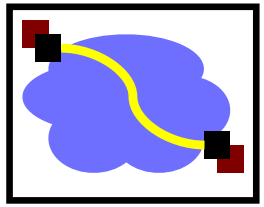
- Mean time to failure (MTTF)
- Mean time to repair (MTTR)
- MTBF = MTTF + MTTR

- Availability = MTTF / (MTTF + MTTR)
  - Suppose OS crashes once per month, takes 10min to reboot.
  - MTTF = 720 hours = 43,200 minutes
    MTTR = 10 minutes
  - Availability = 43200 / 43210 = 0.997 (~“3 nines”)

Disk failure conditional probability distribution - Bathtub curve

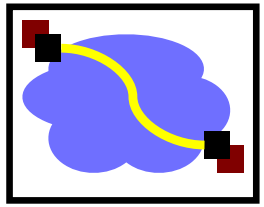


What are our options?



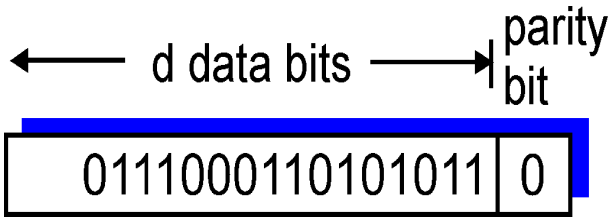
1. Silently return the wrong answer.
2. Detect failure.
3. Correct / mask the failure

Parity Checking

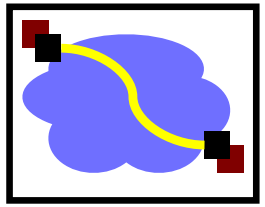


Single Bit Parity:

Detect single bit errors

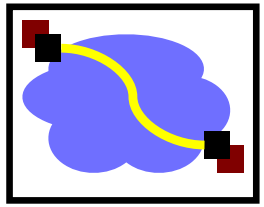


Error Detection - Checksum

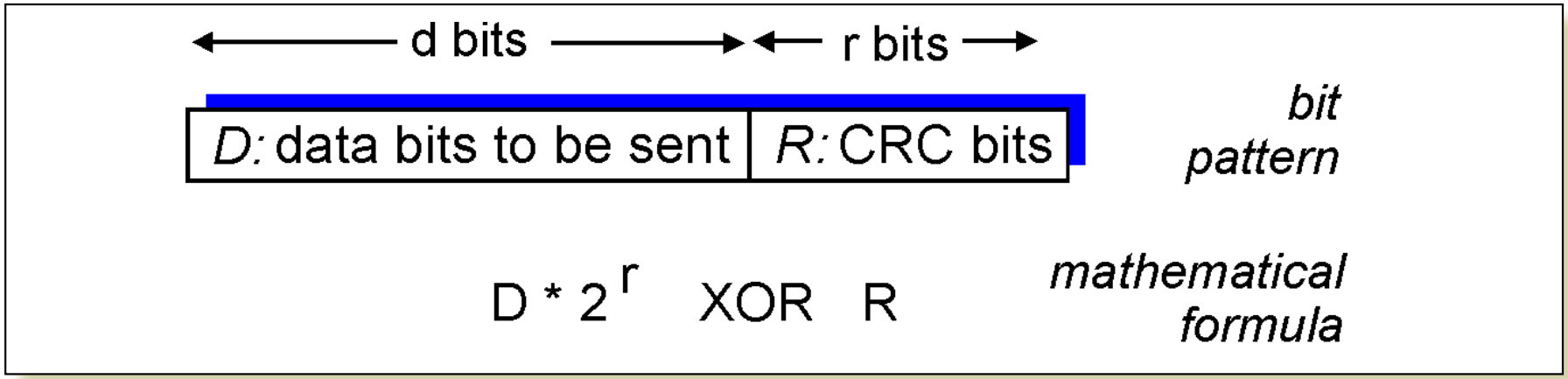


- Used by TCP, UDP, IP, etc..
- Ones complement sum of all words/shorts/bytes in packet
- Simple to implement
- Relatively weak detection
  - Easily tricked by typical error patterns – e.g. bit flips

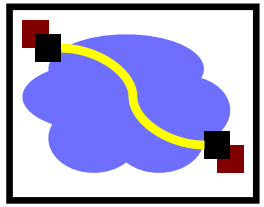
Error Detection – CRC



- View data bits, D, as a binary number
- Choose r+1 bit pattern (generator), G
- Goal: choose r CRC bits, R, such that
  - <D,R> exactly divisible by G (modulo 2)
  - Receiver knows G, divides <D,R> by G. If non-zero remainder: error detected!
  - Can detect all burst errors less than r+1 bits
- Widely used in practice

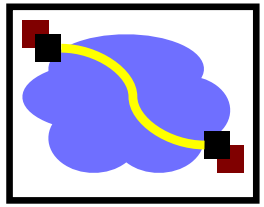


Error Recovery



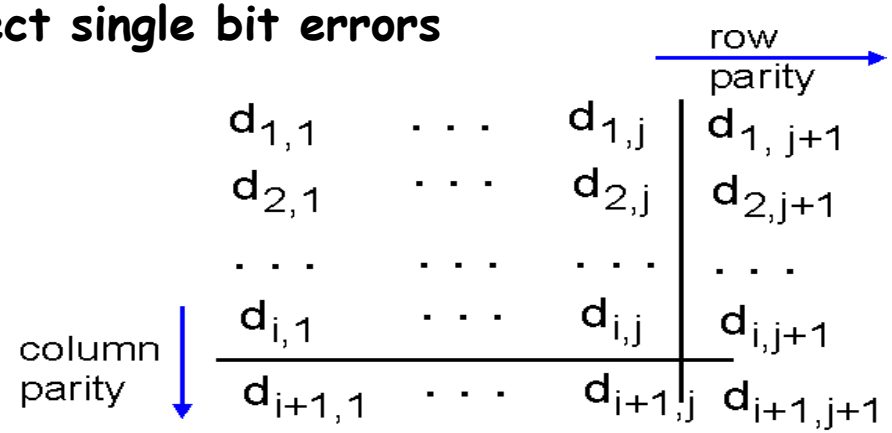
- Two forms of error recovery
  - Redundancy
    - Error Correcting Codes (ECC)
    - Replication/Voting
  - Retry
- ECC
  - Keep encoded redundant data to help repair losses
  - Forward Error Correction (FEC) – send bits in advance
    - Reduces latency of recovery at the cost of bandwidth

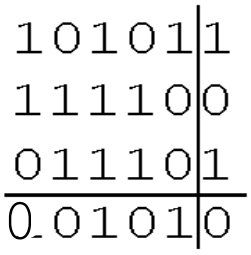
Error Recovery – Error Correcting Codes (ECC)



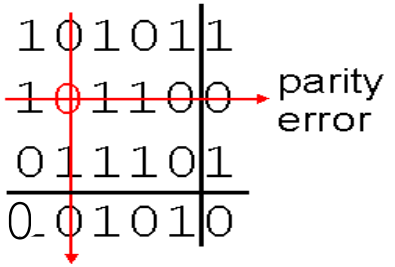
Two Dimensional Bit Parity:

Detect and correct single bit errors



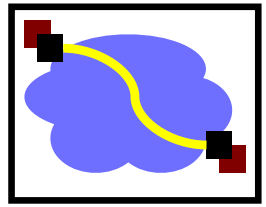


no errors

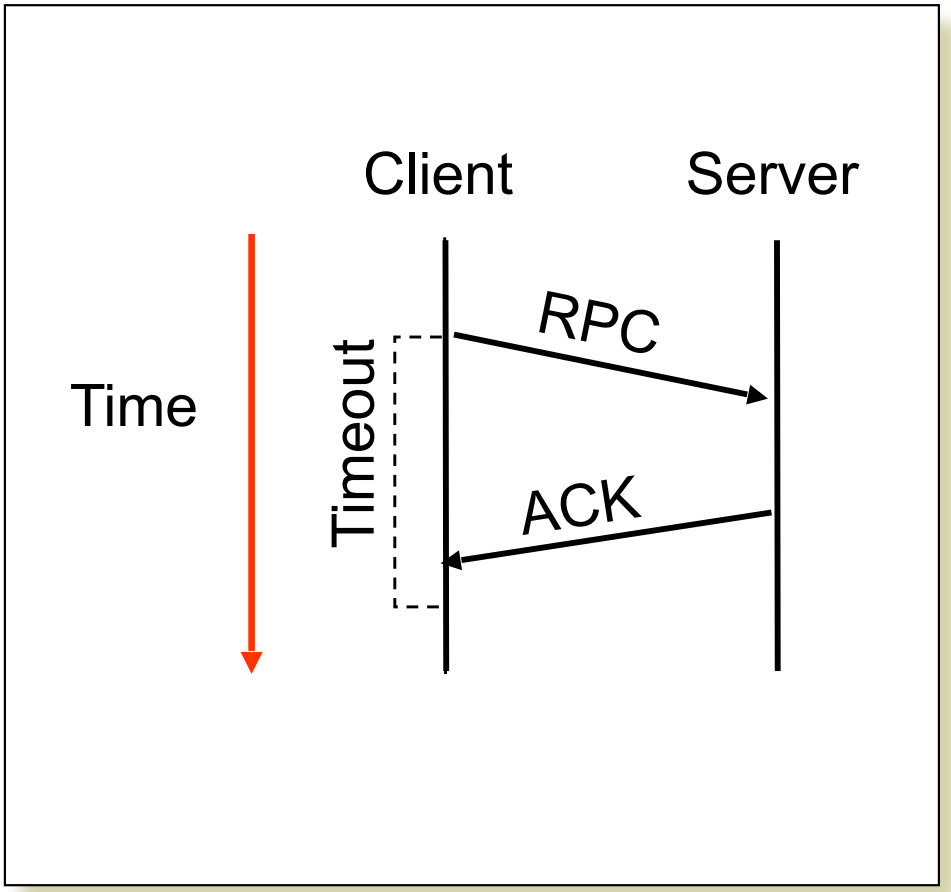


correctable single bit error

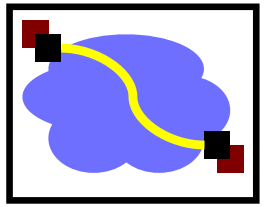
Retry – Network Example



- Sometimes errors are transient
- Need to have error detection mechanism
  - E.g., timeout, parity, chksum
  - No need for majority vote

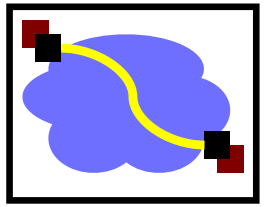


One key question



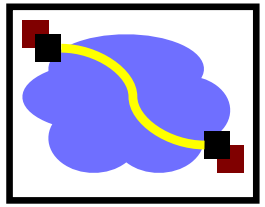
- How correlated are failures?
- Can you assume independence?
  - If the failure probability of a computer in a rack is p,
  - What is p(computer 2 failing) | computer 1 failed?
    - Maybe it's p... or maybe they're both plugged into the same UPS...
- Why is this important?
  - Correlation reduces value of redundancy

Replacement Rates



HPC1		COM1		COM2	
Component	**%**	**Component**	**%**	**Component**	**%**
Hard drive	30.6	Power supply	34.8	Hard drive	49.1
Memory	28.5	Memory	20.1	Motherboard	23.4
Misc/Unk	14.4	Hard drive	18.1	Power supply	10.1
CPU	12.4	Case	11.4	RAID card	4.1
motherboard	4.9	Fan	8	Memory	3.4
Controller	2.9	CPU	2	SCSI cable	2.2
QSW	1.7	SCSI Board	0.6	Fan	2.2
Power supply	1.6	NIC Card	1.2	CPU	2.2
MLB	1	LV Pwr Board	0.6	CD-ROM	0.6
SCSI BP	0.3	CPU heatsink	0.6	Raid Controller	0.6

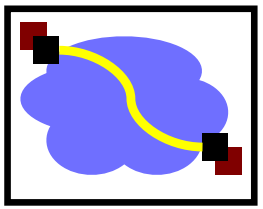
Motivation: Why use multiple disks?

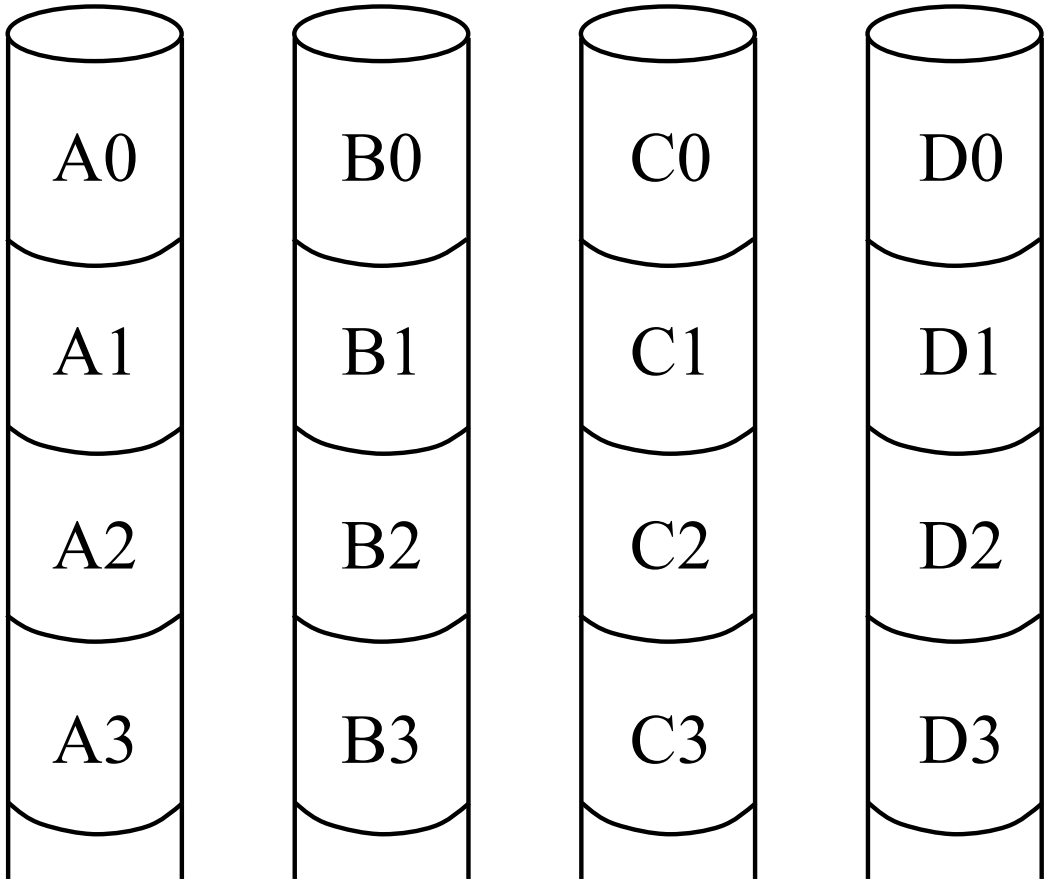


- Capacity
  - More disks allows us to store more data
- Performance
  - Access multiple disks in parallel
  - Each disk can be working on independent read or write
  - Overlap seek and rotational positioning time for all
- Reliability
  - Recover from disk (or single sector) failures
  - Will need to store multiple copies of data to recover

- So, what is the simplest arrangement?

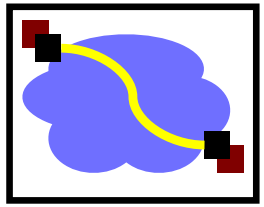
Just a bunch of disks (JBOD)



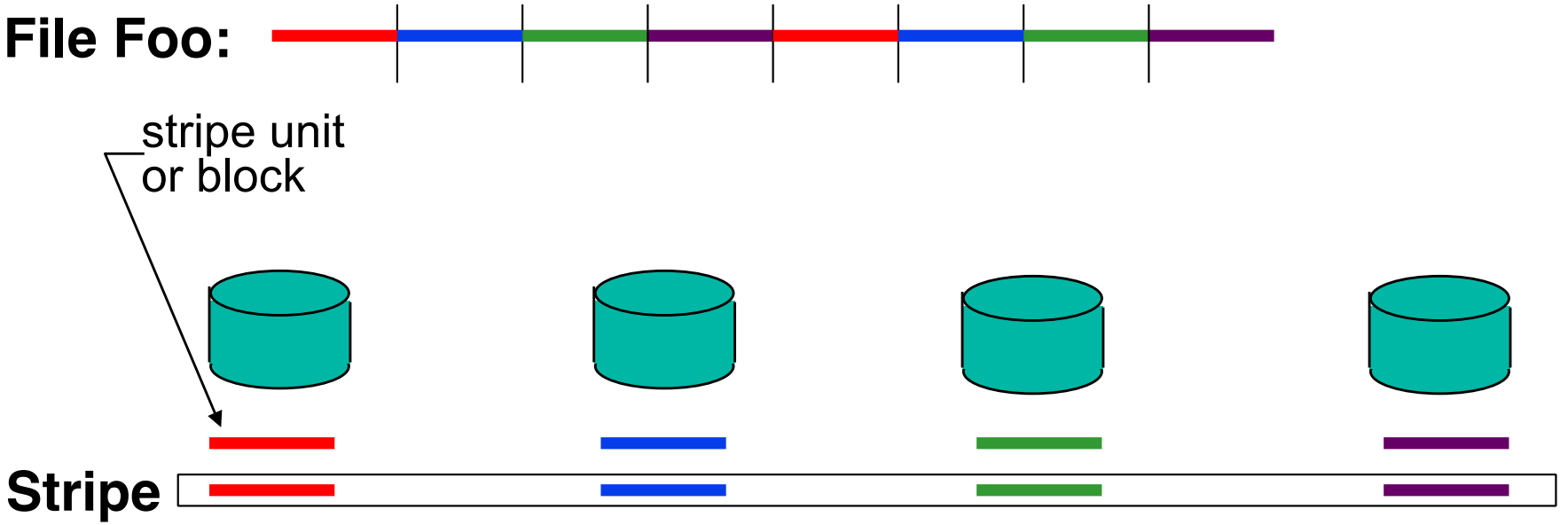


- Yes, it's a goofy name
  - industry really does sell "JBOD enclosures"

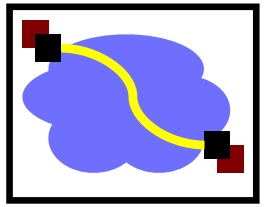
Disk Striping



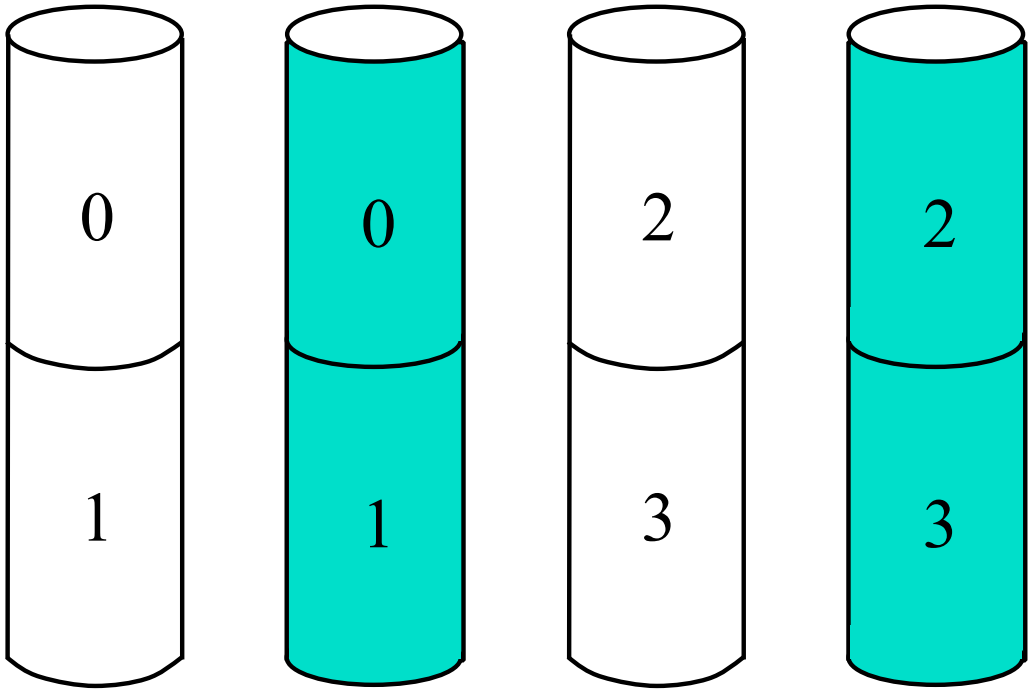
- Interleave data across multiple disks
  - Large file streaming can enjoy parallel transfers
  - High throughput requests can enjoy thorough load balancing
    - If blocks of hot files equally likely on all disks (really?)



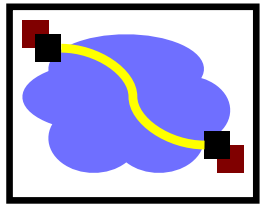
Redundancy via replicas



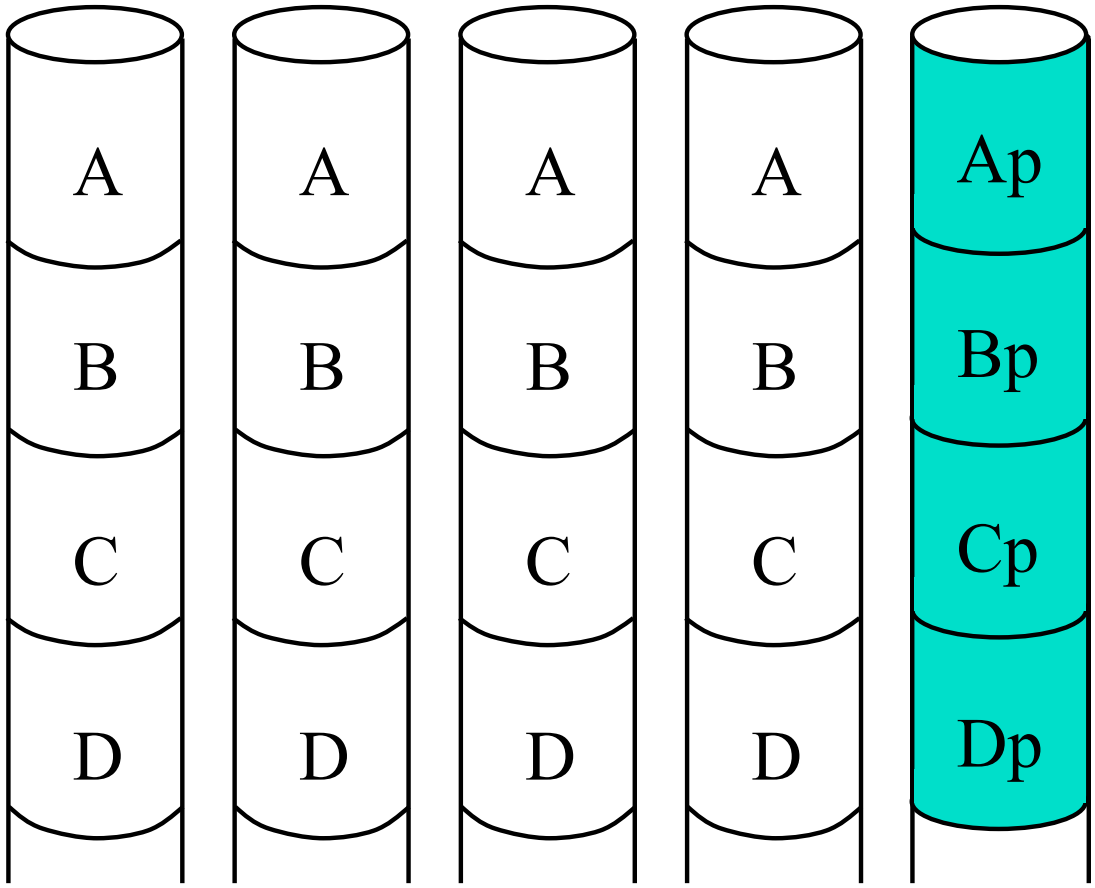
- Two (or more) copies
  - mirroring, shadowing, duplexing, etc.
- Write both, read either



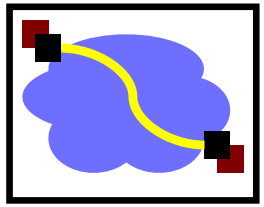
Simplest approach: Parity Disk

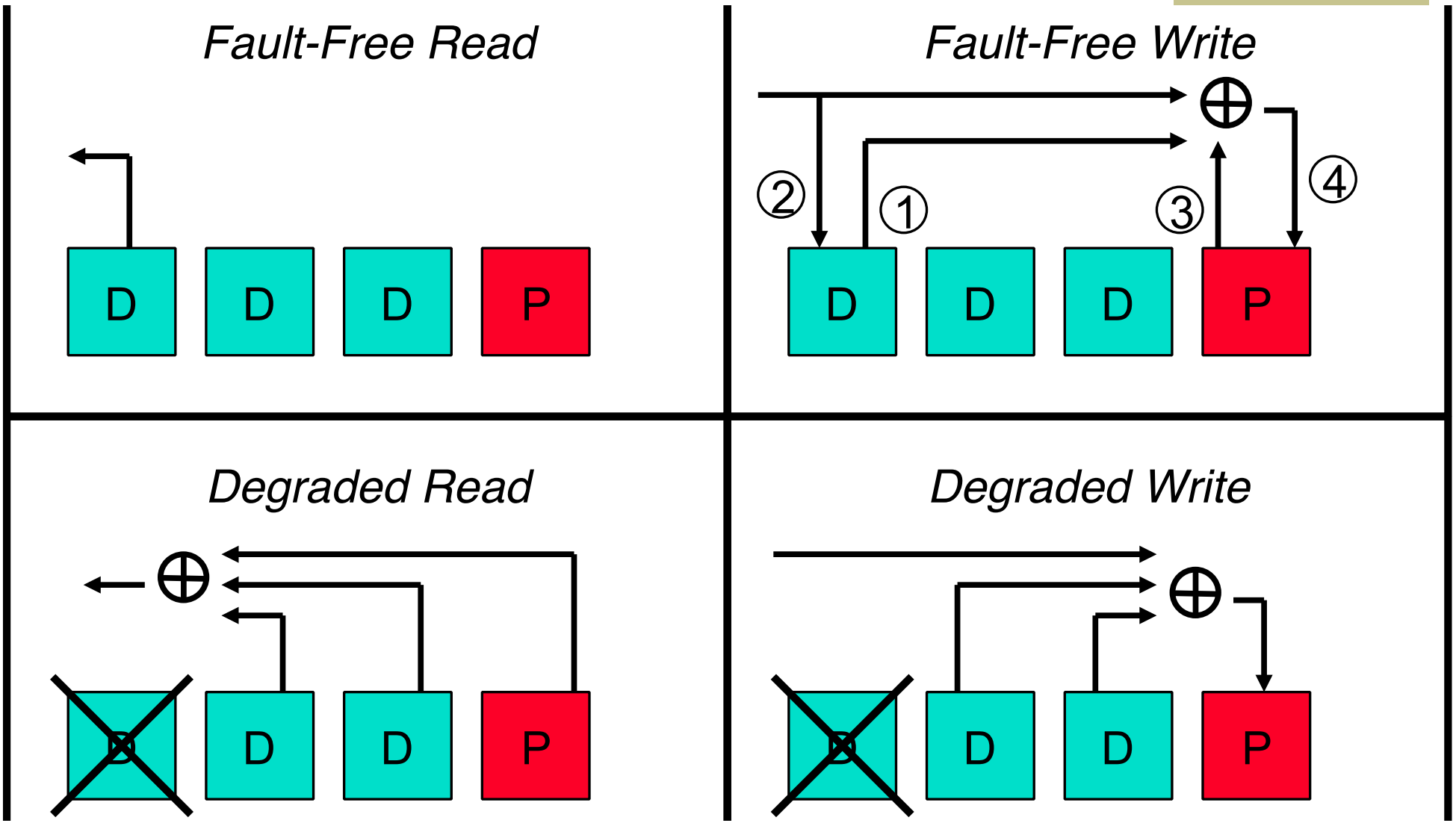


- Capacity: one extra disk needed per stripe

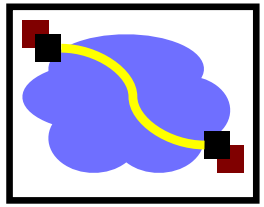


Updating and using the parity

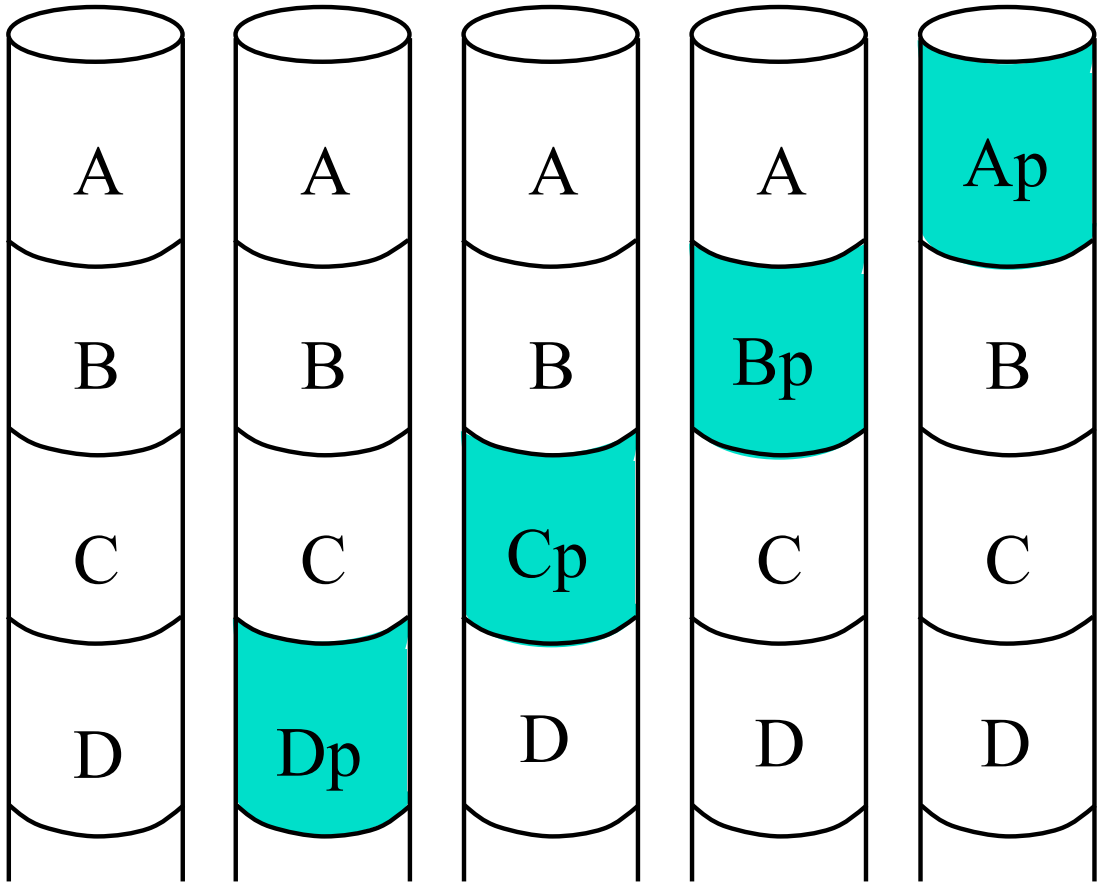




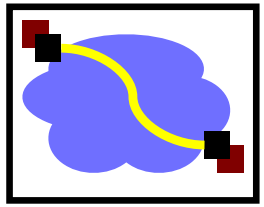
Solution: Striping the Parity



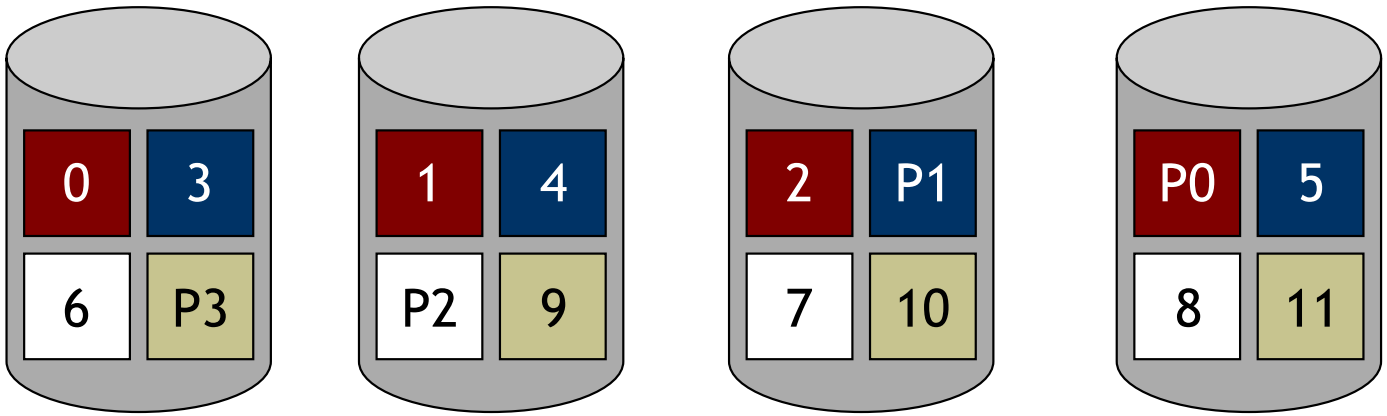
- Removes parity disk bottleneck



RAID-5: Rotated Parity

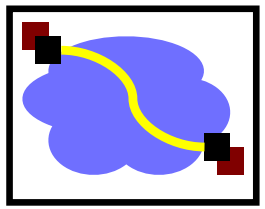


Rotate location of parity across all disks



- Capacity:
- Reliability:
- Performance:
  - Reads:
  - Writes:
  - Still requires 4 I/Os per write, but not always to same parity disk

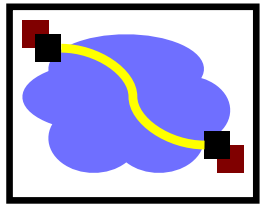
Comparison



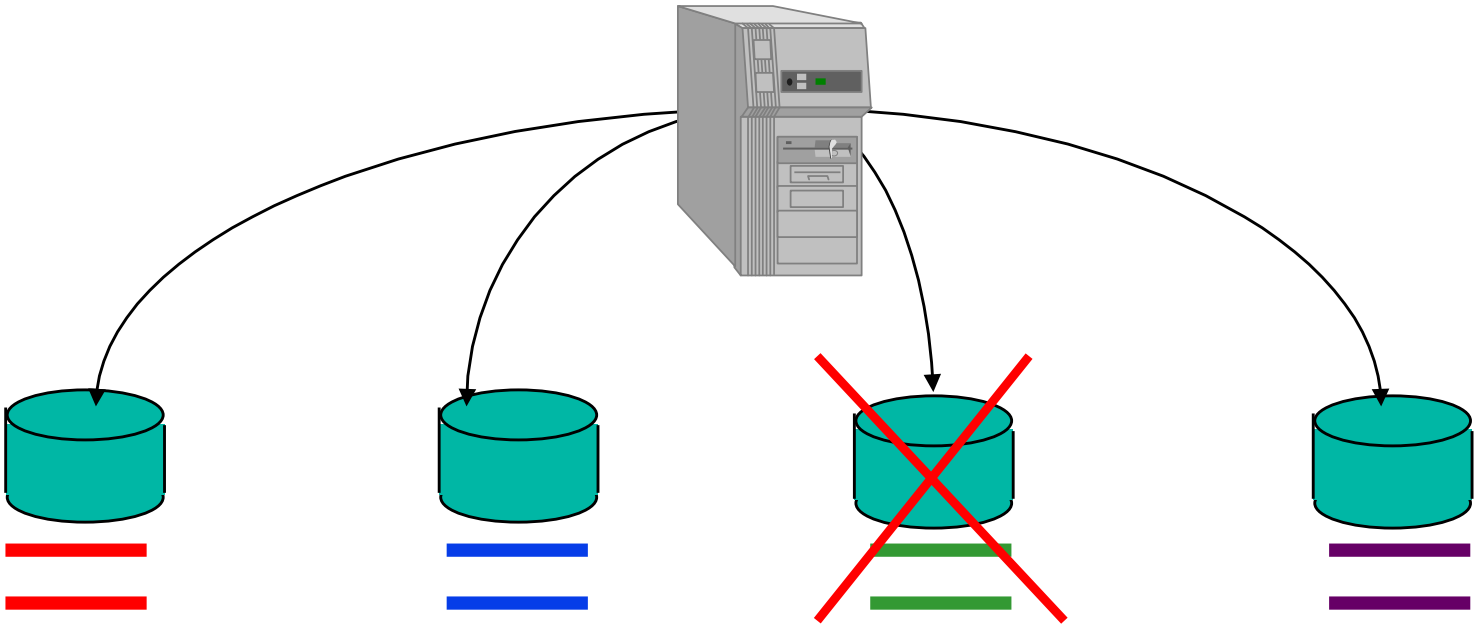
	RAID-0	RAID-1	RAID-4	RAID-5
Capacity	N	N / 2	N − 1	N − 1
Reliability	0	1 (for sure) $\frac{N}{2}$ (if lucky)	1	1
Throughput				
Sequential Read	N · S	N x R	(N − 1) · S	(N − 1) · S
Sequential Write	N · S	(N / 2) · S	(N − 1) · S	(N − 1) · S
Random Read	N · R	N · R	(N − 1) · R	N · R
Random Write	N · R	(N / 2) · R	$\frac{1}{2}$ · R	$\frac{N}{4}$ R
Latency				
Read	D	D	D	D
Write	D	D	2D	2D

Key takeaways: writes are expensive, small writes are really expensive! File systems may help (see LFS)

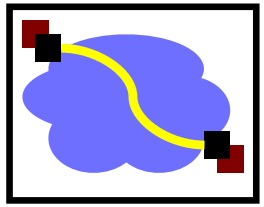
How often are failures?



- MTBF (Mean Time Between Failures)
  - $MTBF_{disk}$ ~ 1,200,00 hours (~136 years, <1% per year)
- $MTBF_{mutli\text{-}disk\ system}$ = mean time to first disk failure
  - which is $MTBF_{disk}$ / (number of disks)
  - For a striped array of 200 drives
  - $MTBF_{array}$ = 136 years / 200 drives = 0.65 years

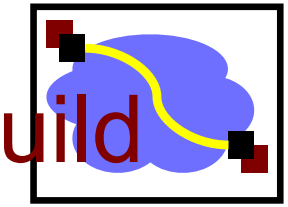


Reliability without rebuild



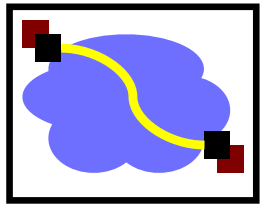
- 200 data drives with $MTBF_{drive}$
  - $MTTDL_{array} = MTBF_{drive} / 200$
- Add 200 drives and do mirroring
  - $MTBF_{pair} = (MTBF_{drive} / 2) + MTBF_{drive} = 1.5 * MTBF_{drive}$
  - $MTTDL_{array} = MTBF_{pair} / 200 = MTBF_{drive} / 133$
- Add 50 drives, each with parity across 4 data disks
  - $MTBF_{set} = (MTBF_{drive} / 5) + (MTBF_{drive} / 4) = 0.45 * MTBF_{drive}$
  - $MTTDL_{array} = MTBF_{set} / 50 = MTBF_{drive} / 111$
    - approximate see note

Reliability consequences of adding rebuild



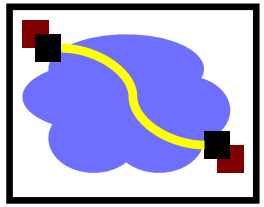
- No data loss, if fast enough
  - That is, if first failure fixed before second one happens
- New math is...
  - $MTTDL_{array} = MTBF_{firstdrive} * (1 / \text{prob of } 2^{nd} \text{ failure before repair})$
  - ... which is $MTTR_{drive} / MTBF_{seconddrive}$
- For mirroring
  - $MTBF_{pair} = (MTBF_{drive} / 2) * (MTBF_{drive} / MTTR_{drive})$
- For 5-disk parity-protected arrays
  - $MTBF_{set} = (MTBF_{drive} / 5) * ((MTBF_{drive} / 4) / MTTR_{drive})$

Three modes of operation



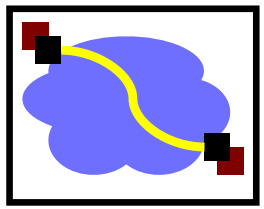
- Normal mode
  - everything working; maximum efficiency
- Degraded mode
  - some disk unavailable
  - must use degraded mode operations
- Rebuild mode
  - reconstructing lost disk's contents onto spare
  - degraded mode operations plus competition with rebuild

Byzantine Fault Tolerance



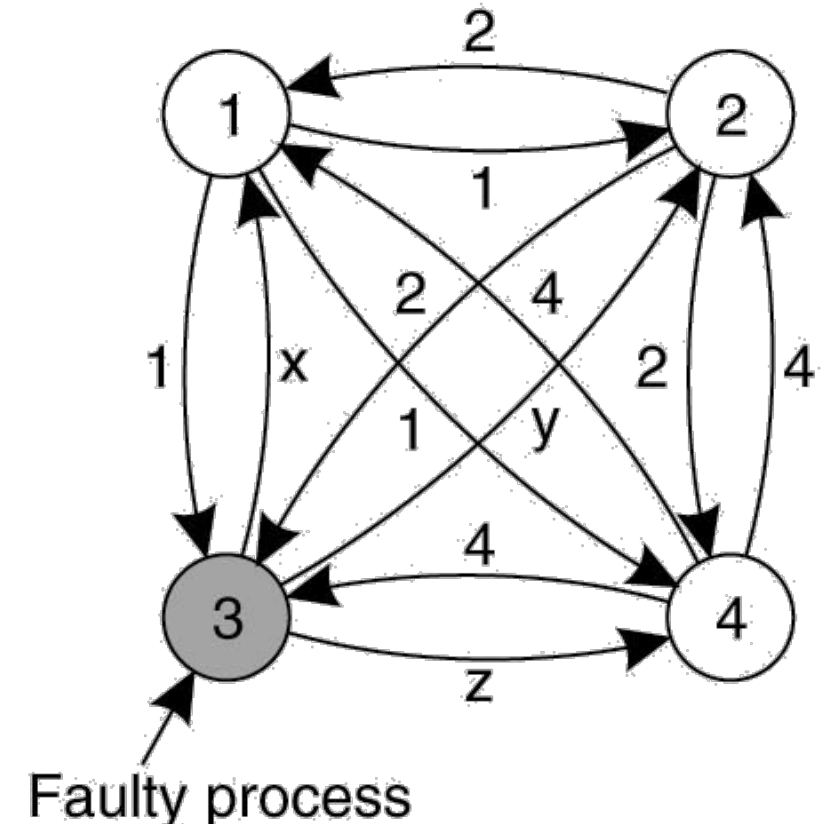
- Terminology & Background
- Byzantine Fault Tolerance (Lamport)
- Async. BFT (Liskov)

Agreement in Faulty Systems - 5



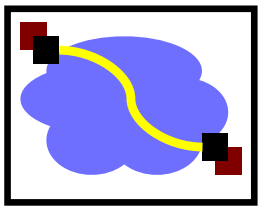
System of N processes, where each process *i* will provide a value v_i to each other. Some number of these processes may be incorrect (or malicious)

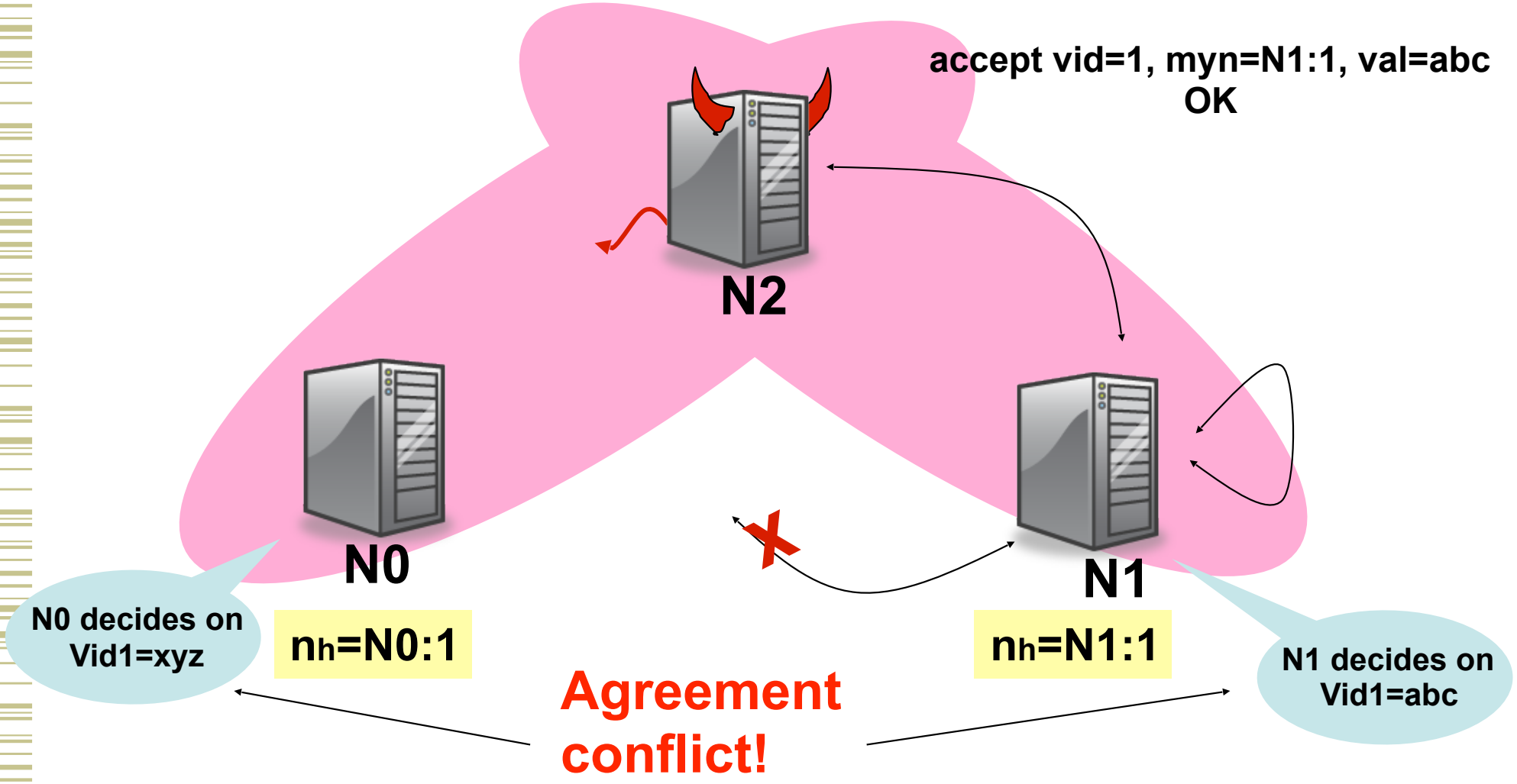
Goal: Each process learn the true values sent by each of the correct processes



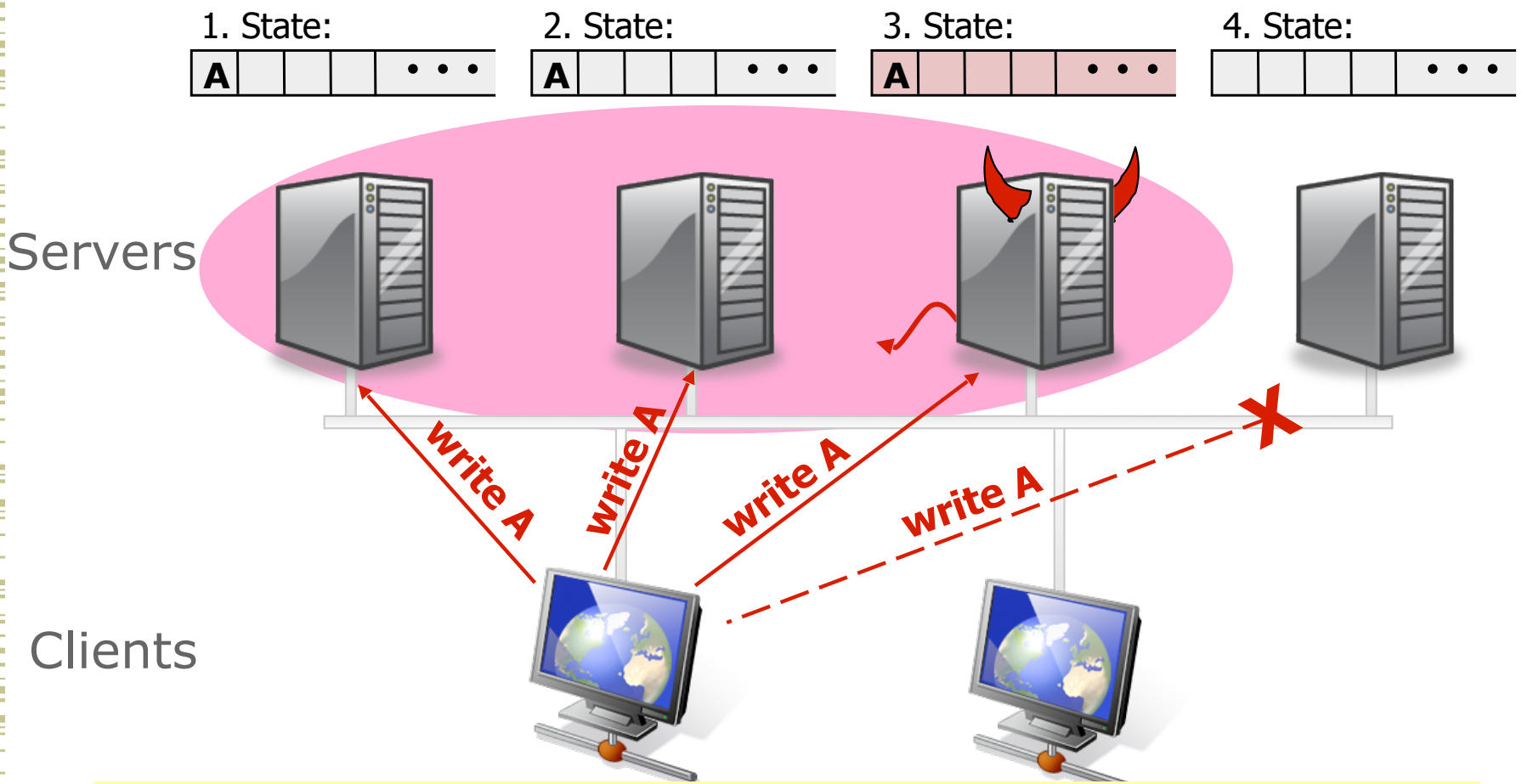
The Byzantine agreement problem for three nonfaulty and one faulty process.

Paxos under Byzantine faults



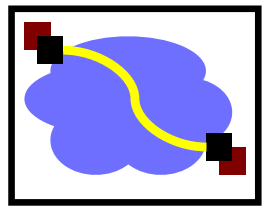


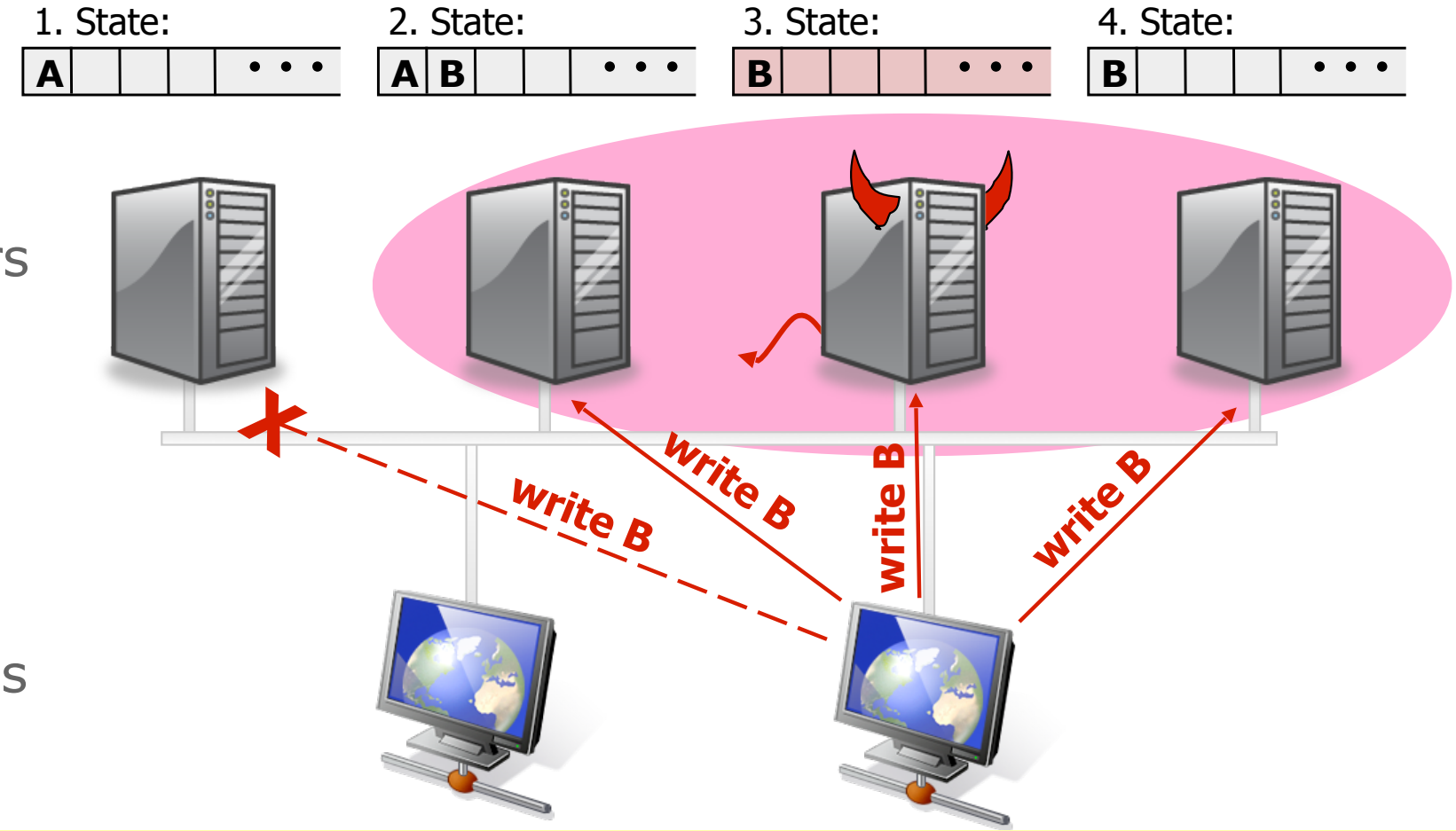
BFT requires a 2f+1 quorum out of 3f+1 nodes



For liveness, the quorum size must be at most N - f

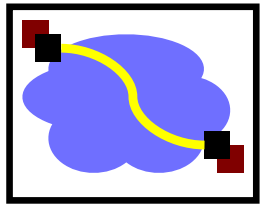
BFT Quorums





For correctness, any two quorums must intersect at least one honest node: (N-f) + (N-f) - N >= f+1 ➡ N >= 3f+1

Distributed systems

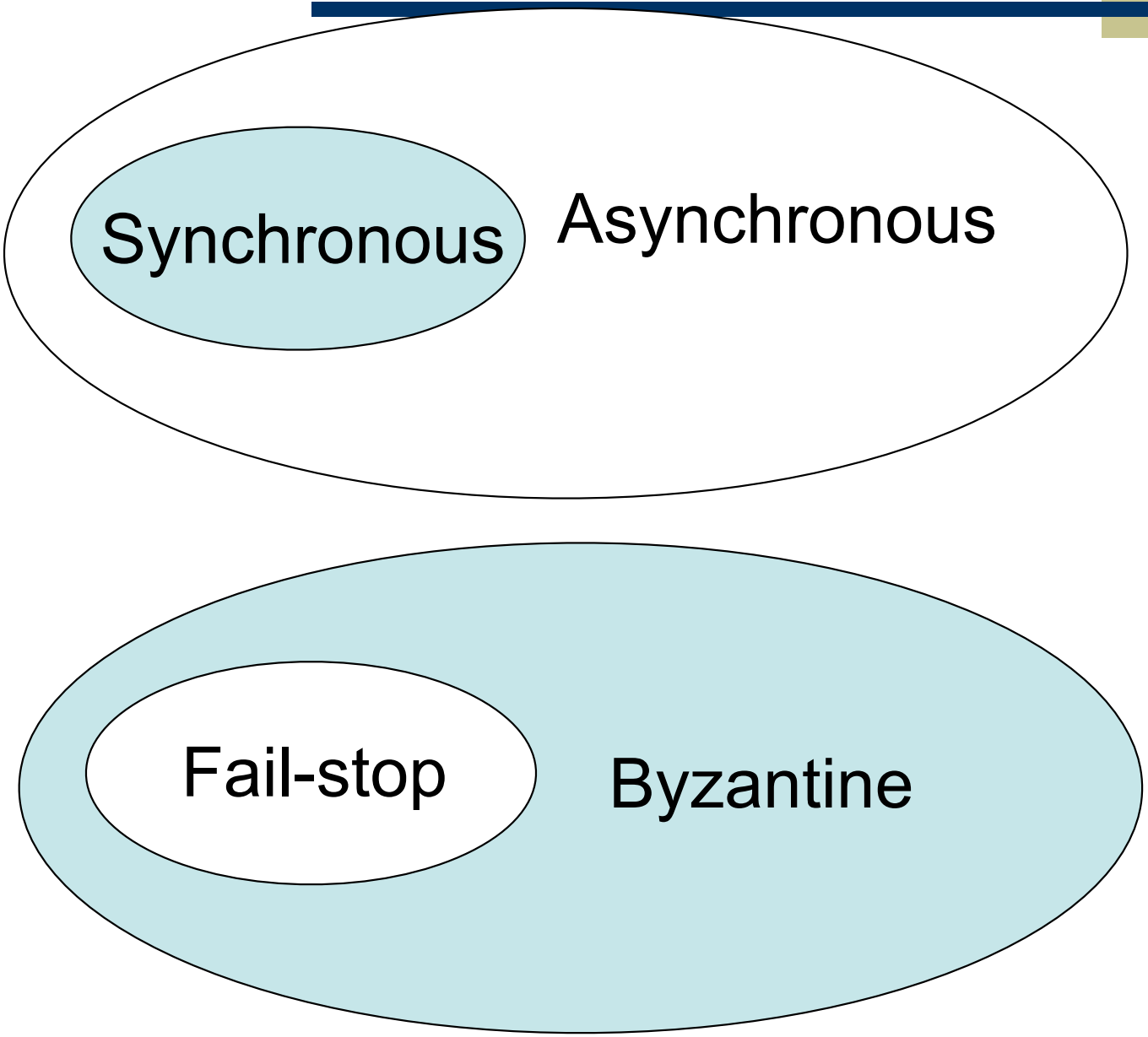


- FLP impossibility: Async consensus may not terminate
  - Sketch of proof: System starts in "bivalent" state (may decide 0 or 1). At some point, the system is one message away from deciding on 0 or 1. If that message is delayed, another message may move the system away from deciding.
  - Holds even when servers can only crash (not Byzantine)!
  - Hence, protocol cannot always be live (but there exist randomized BFT variants that are probably live)

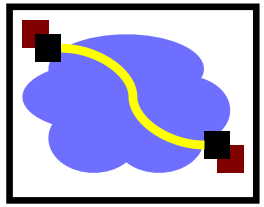
[See Fischer, M. J., Lynch, N. A., and Paterson, M. S. 1985. Impossibility of distributed consensus with one faulty process. J. ACM 32, 2 (Apr. 1985), 374-382.]

In the system Fischer, Lynch, and Paterson studied, messages were unordered, communication was unbounded, and processors were asynchronous.

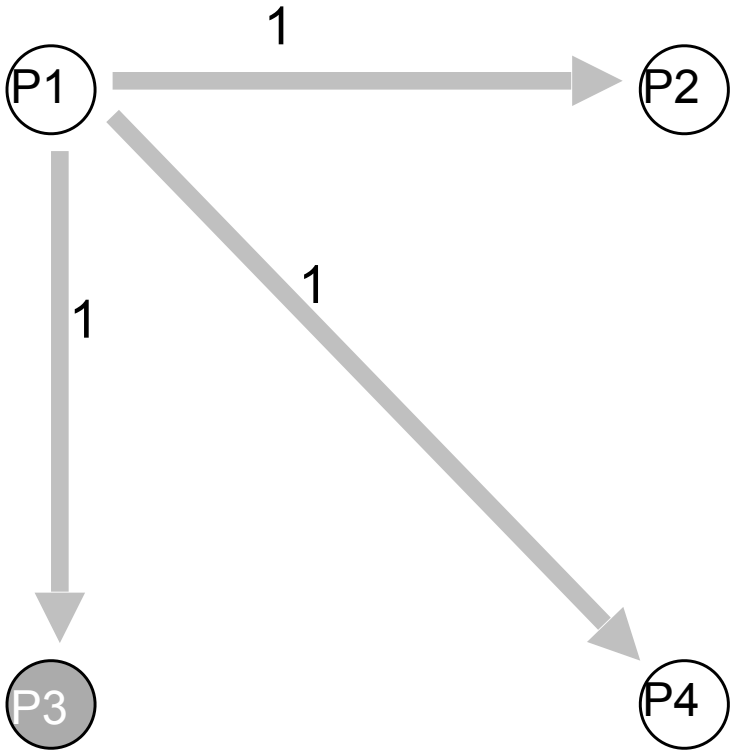
Synchronous, Byzantine world



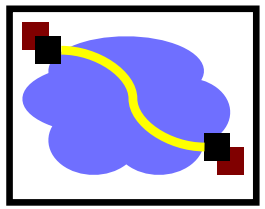
Byzantine General Problem Example - 1



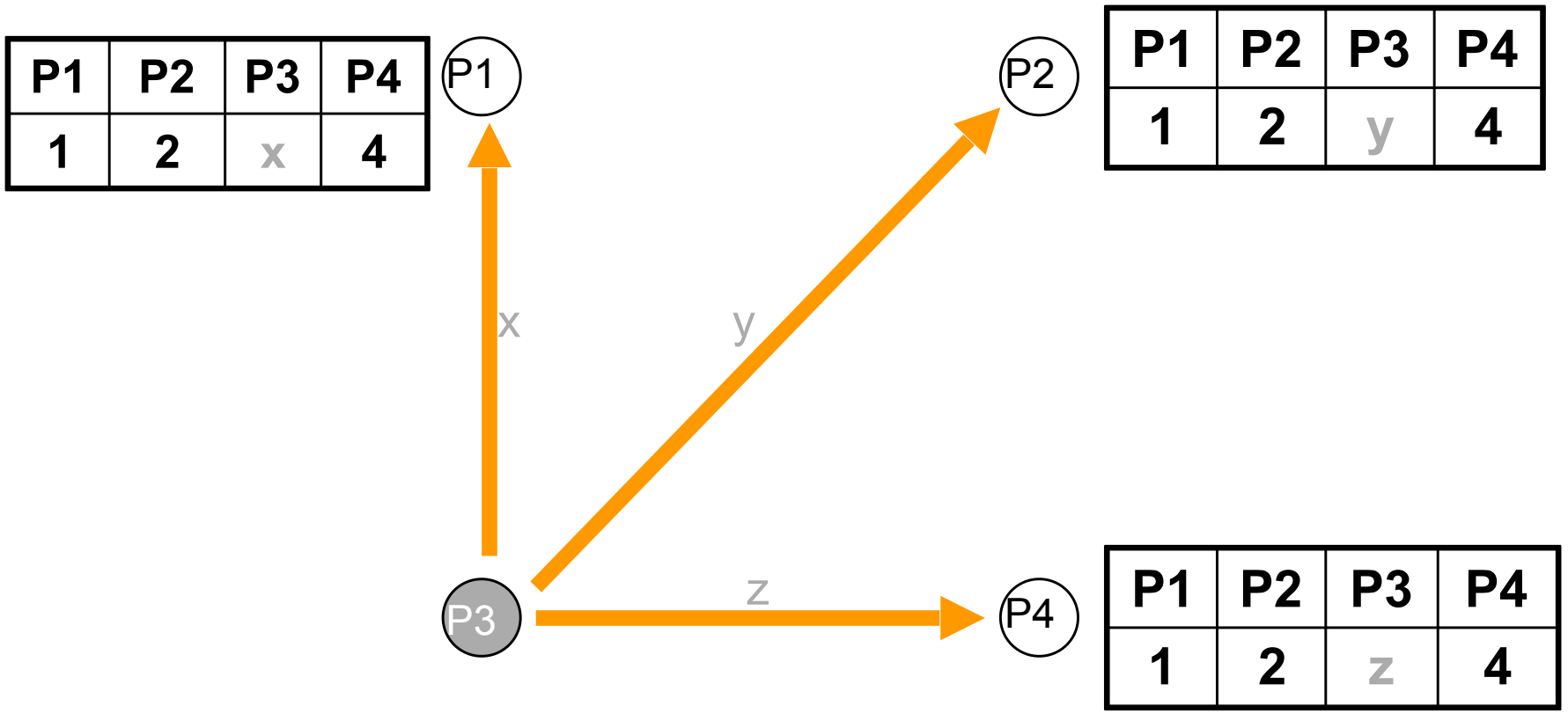
- Phase 1: Generals announce their troop strengths to each other



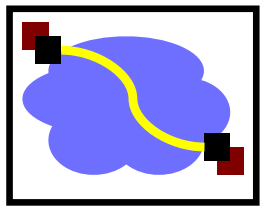
Byzantine General Problem Example - 4



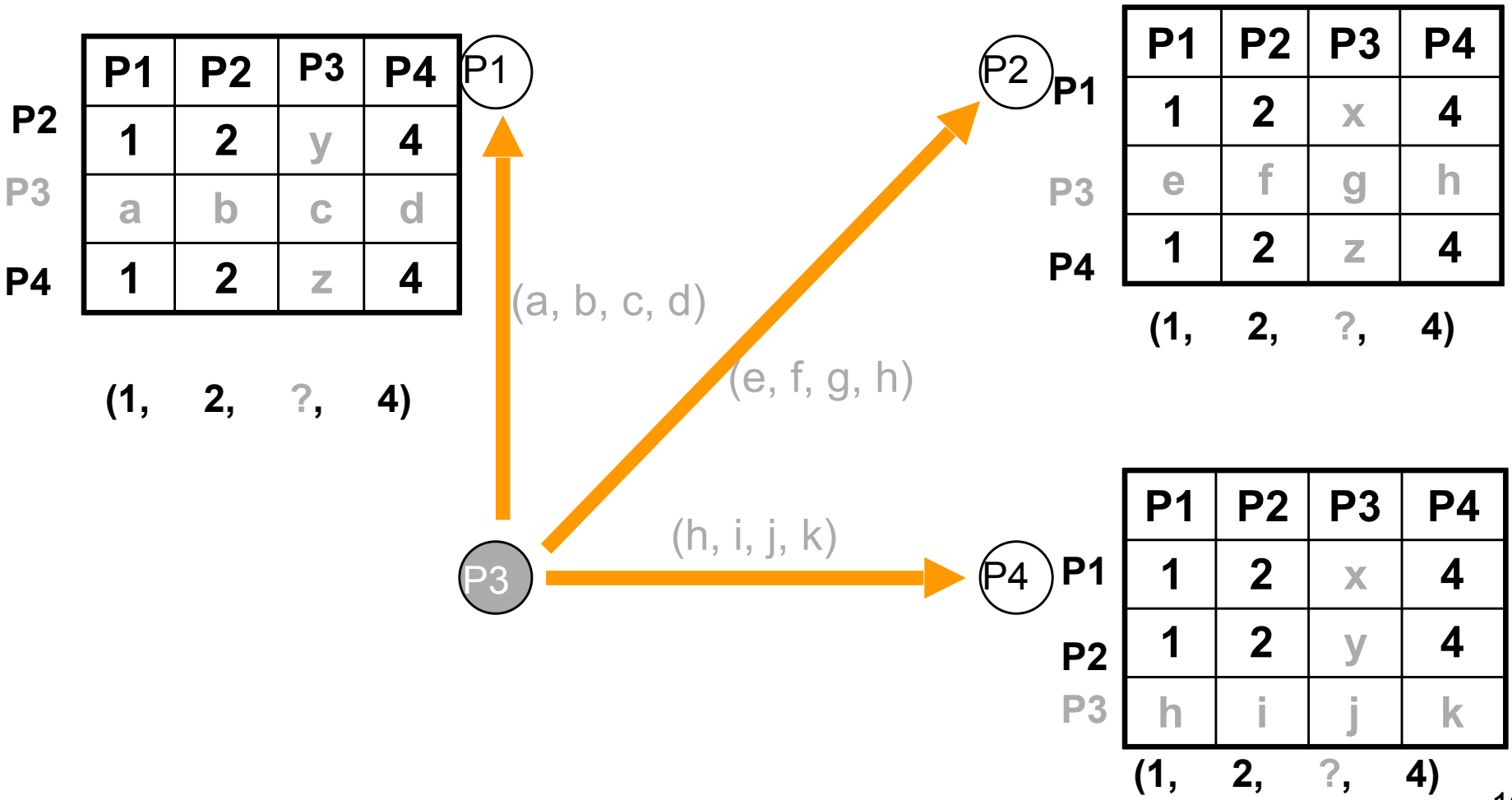
- Phase 2: Each general construct a vector with all troops



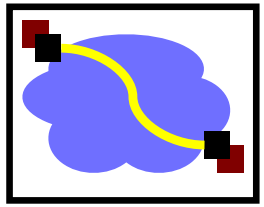
Byzantine General Problem Example - 5

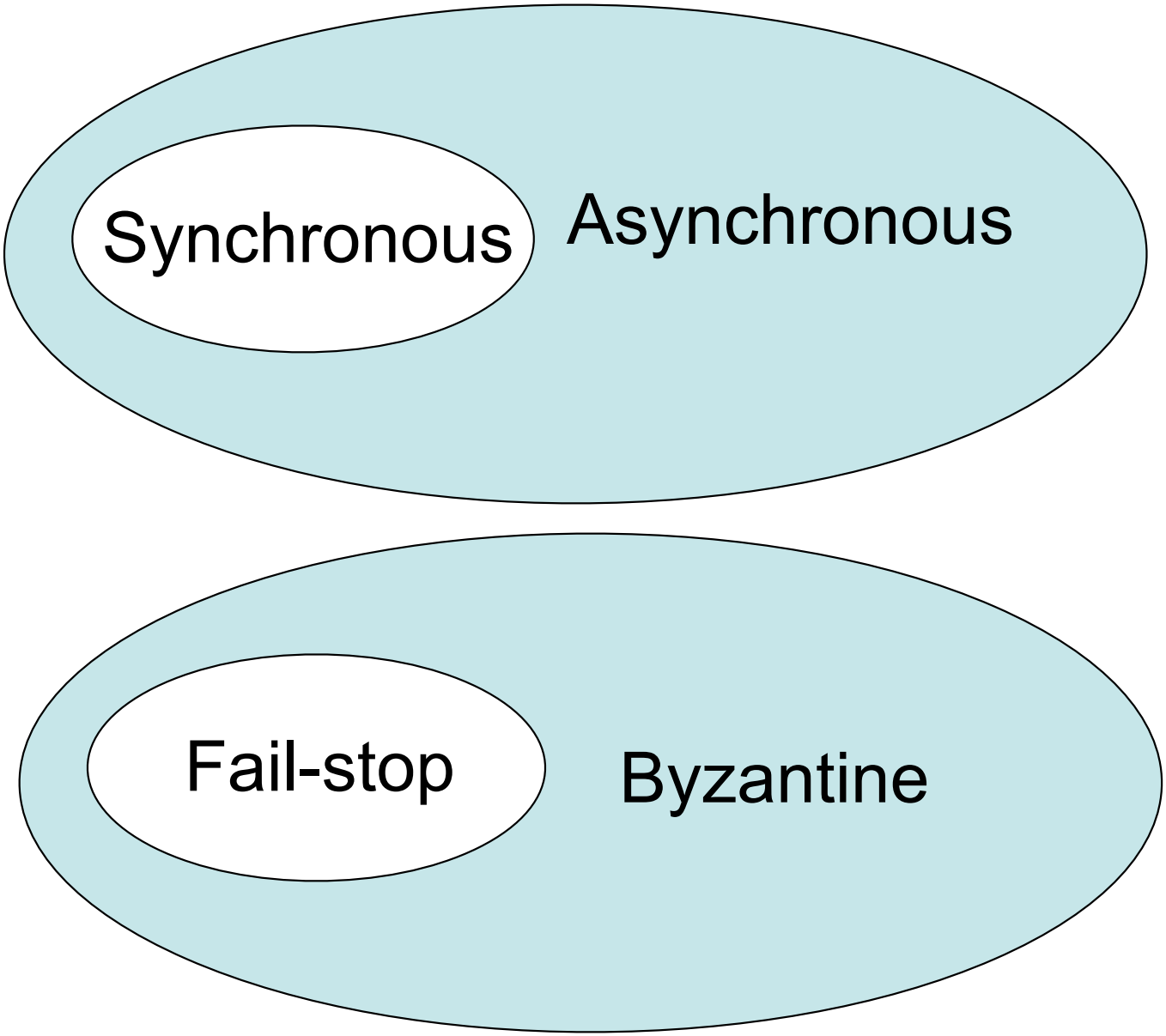


- Phase 3,4: Generals send their vectors to each other and compute majority voting

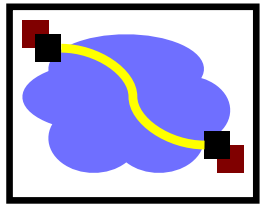


Practical Byzantine Fault Tolerance:Asynchronous, Byzantine



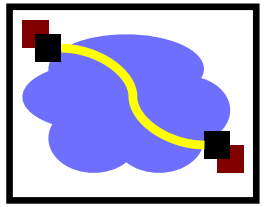


PBFT ideas



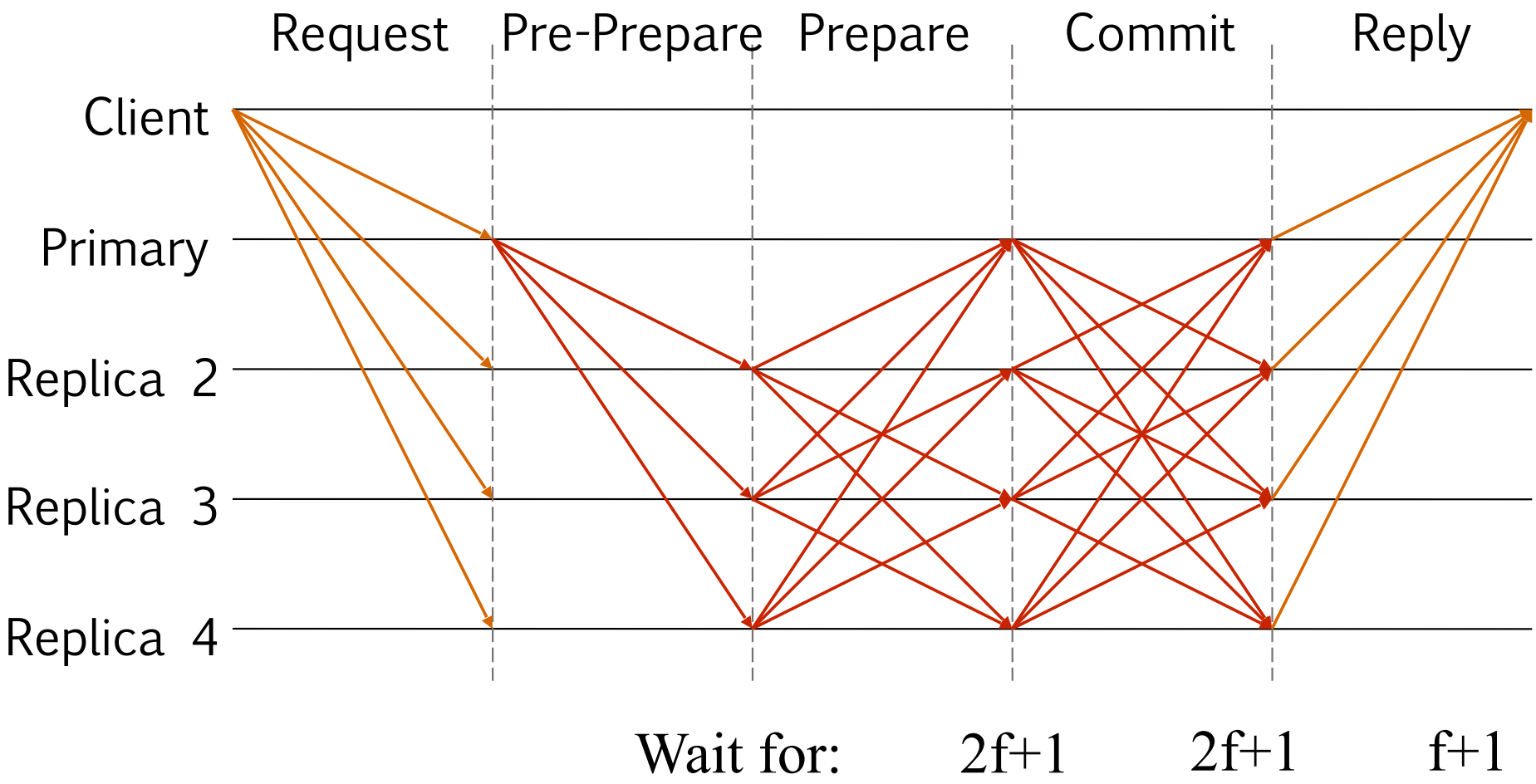
- PBFT, “Practical Byzantine Fault Tolerance”, M. Castro and B. Liskov, SOSP 1999
- Replicate service across many nodes
  - Assumption: only a small fraction of nodes are Byzantine
  - Rely on a super-majority of votes to decide on correct computation.
  - Makes some weak synchrony (message delay) assumptions to ensure liveness
    - Would violate FLP impossibility otherwise
- PBFT property: tolerates <=f failures using a RSM with 3f+1 replicas

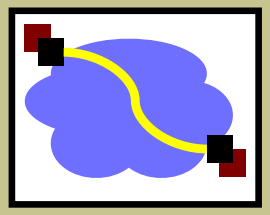
PBFT main ideas



- Static configuration (same 3f+1 nodes)
- To deal with malicious primary
  - Use a 3-phase protocol to agree on sequence number
- To deal with loss of agreement
  - Use a bigger quorum (2f+1 out of 3f+1 nodes)
- Need to authenticate communications

PBFT


Request
Pre-Prepare
Prepare
Commit
Reply
Client
Primary
Replica 2
Replica 3
Replica 4
Wait for:
2f+1
2f+1
f+1



Security